Домашние задание 4

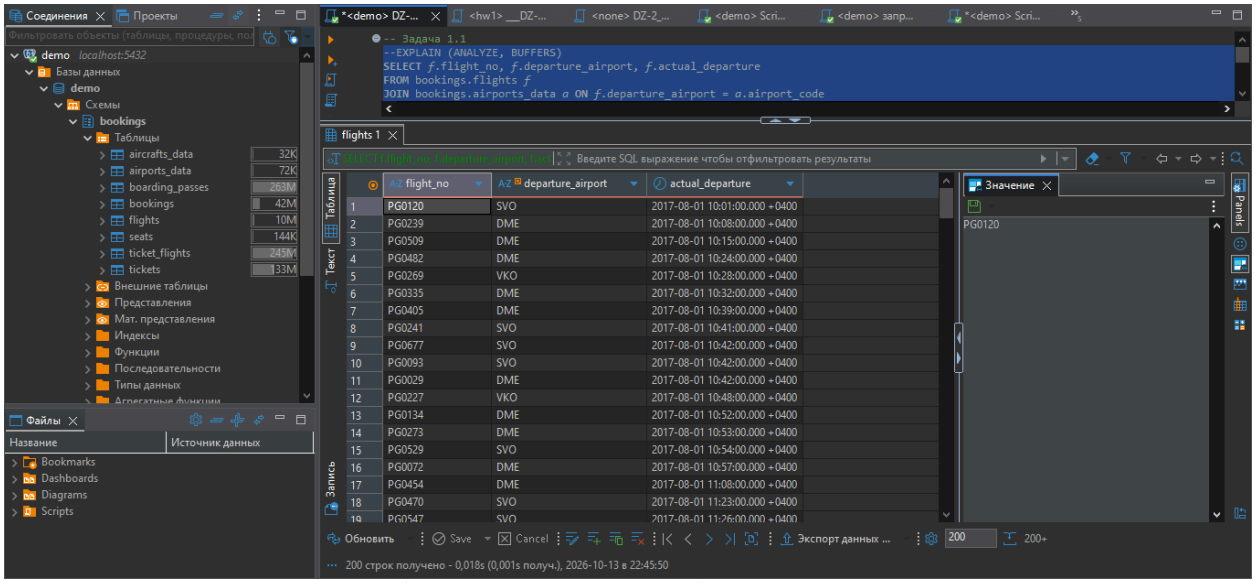
Выполнила: Яицкая Ангелина группа ДЭ 16-25

Запросы с семинара:

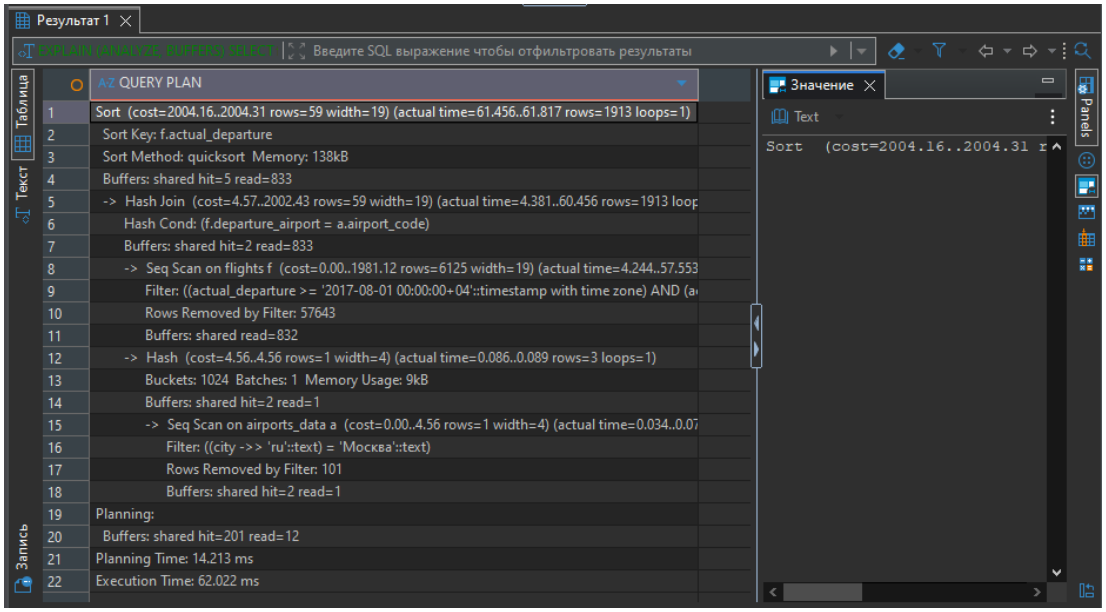
Задача 1.1 Выбрать все выполненные рейсы (status = 'Arrived') из московских аэропортов за август 2017 года: номер рейса, аэропорт вылета, фактическое время вылета. Отсортировать по возрастанию времени вылета.

```
--EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_no, f.departure_airport, f.actual_departure
FROM bookings.flights f
JOIN bookings.airports_data a ON f.departure_airport = a.airport_code
WHERE f.status = 'Arrived'
  AND a.city ->> 'ru' = 'Москва'
  AND f.actual_departure >= '2017-08-01'
  AND f.actual_departure <  '2017-09-01'
ORDER BY f.actual_departure ASC;
```

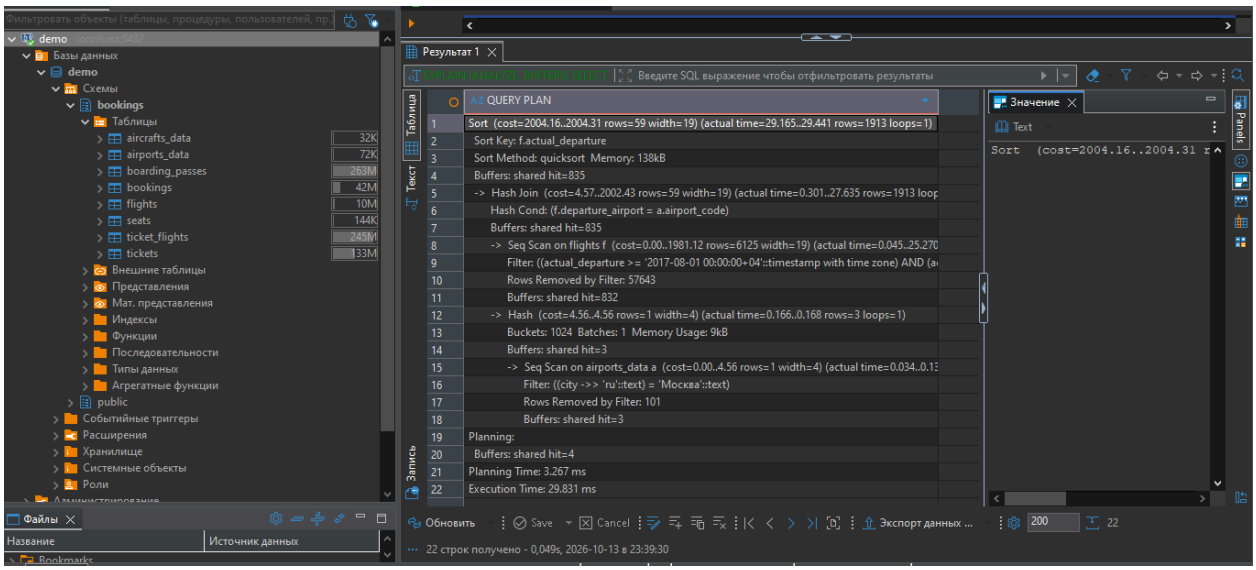
Вывод:



1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



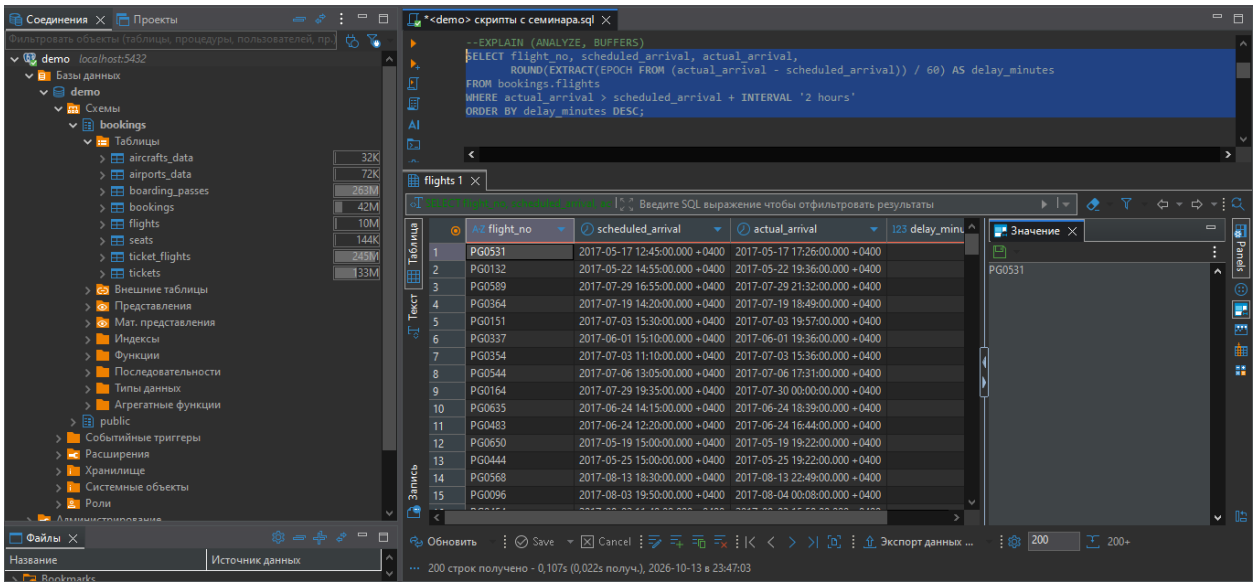
Аналитическое резюме:

План Seq Scan по flights + Seq Scan по airports_data, соединение Hash Join, сортировка quicksort в памяти. Первый запуск read=832 (всё с диска, 62 мс), второй hit=835, read=0 (всё из кэша, 29.8 мс). Индекс: по status и actual_departure индексов в базе нет, поэтому Seq Scan - единственный вариант.

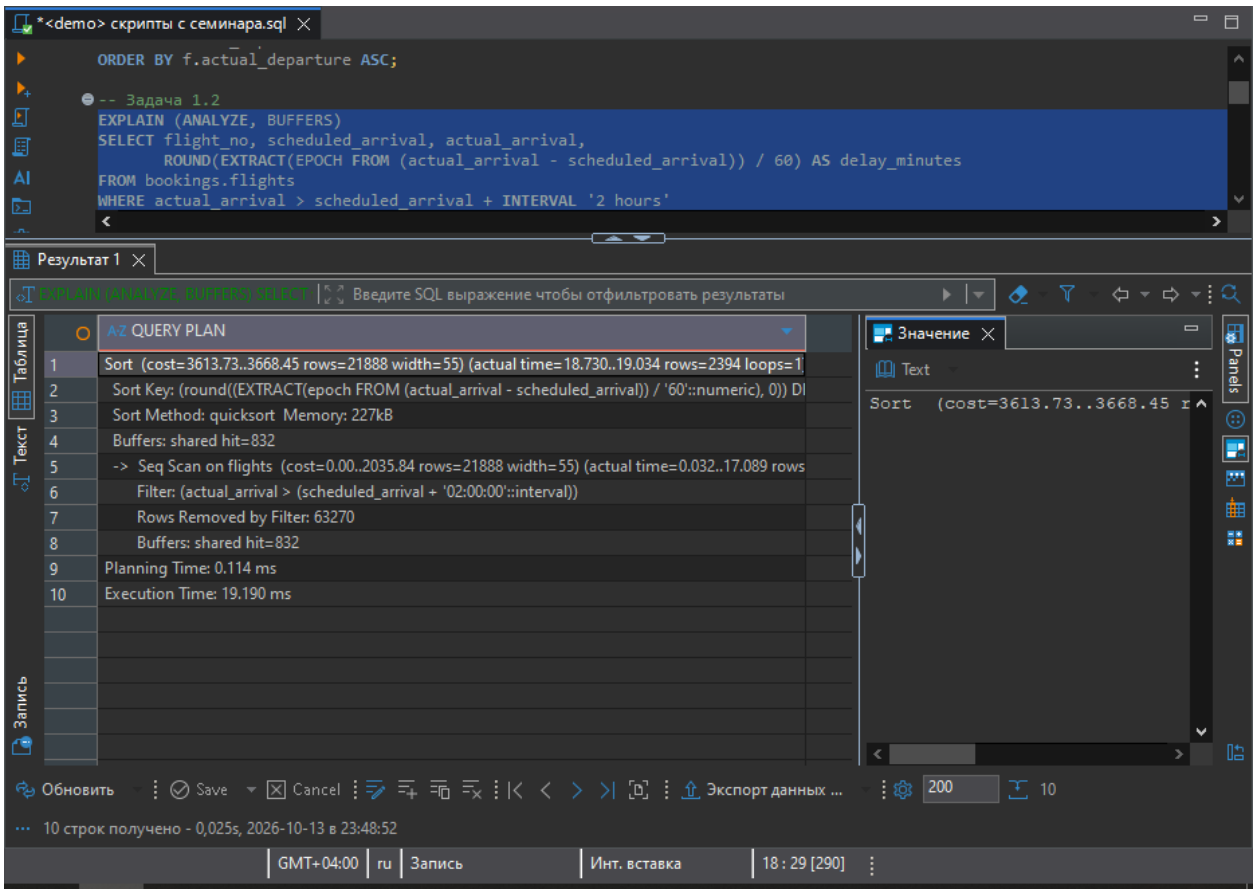
Задача 1.2. Найти рейсы, прибывшие с опозданием более чем на 2 часа: номер рейса, плановое и фактическое время прибытия, опоздание в минутах. Самые большие опоздания - сверху.

```
--EXPLAIN (ANALYZE, BUFFERS)
SELECT flight_no, scheduled_arrival, actual_arrival,
    ROUND(EXTRACT(EPOCH FROM (actual_arrival - scheduled_arrival)) / 60) AS delay_minutes
FROM bookings.flights
WHERE actual_arrival > scheduled_arrival + INTERVAL '2 hours'
ORDER BY delay_minutes DESC;
```

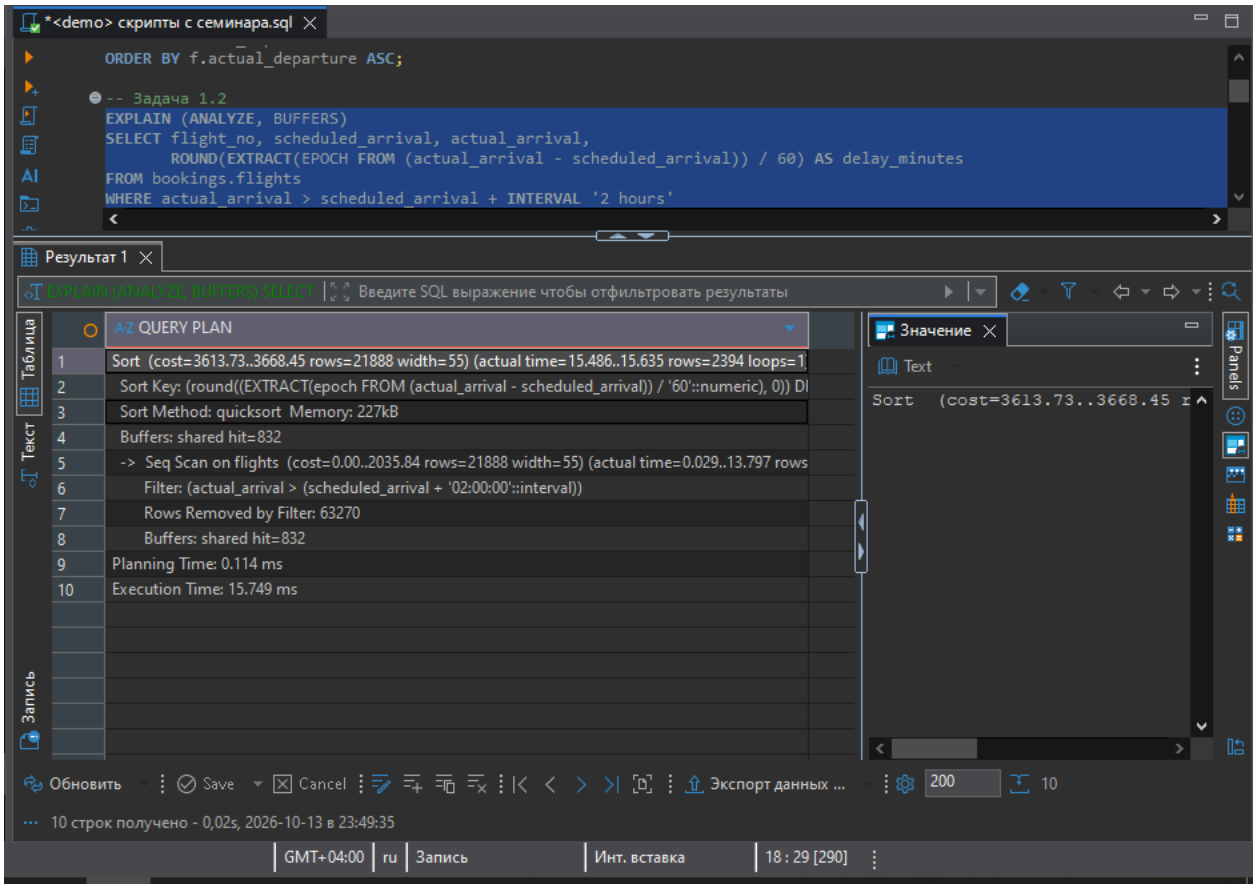
Вывод:



1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



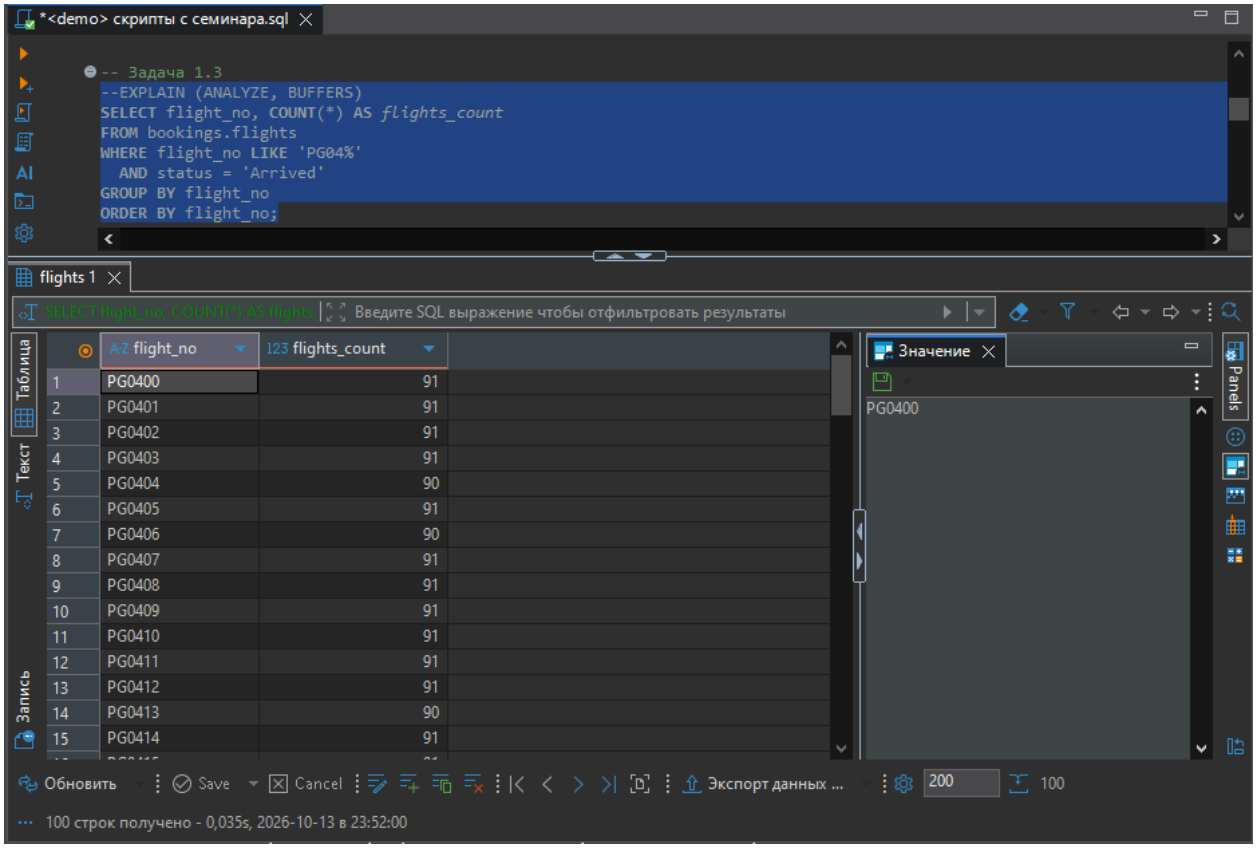
Аналитическое резюме:

План Seq Scan по flights, сортировка quicksort в памяти (227 кБ), соединений нет. Первый запуск hit=832, read=0 (всё из кэша, 19.2 мс), второй hit=832, read=0 (всё из кэша, 15.7 мс). Индекс: по actual_arrival и scheduled_arrival индексов нет, а предикат actual_arrival > scheduled_arrival + INTERVAL не sargable (сравнение двух колонок с арифметикой), поэтому Seq Scan - единственный вариант.

Задача 1.3. Рейсы с номерами, начинающимися на PG04 (номера диапазона PG0400–PG0499): сколько выполнено рейсов каждым номером.

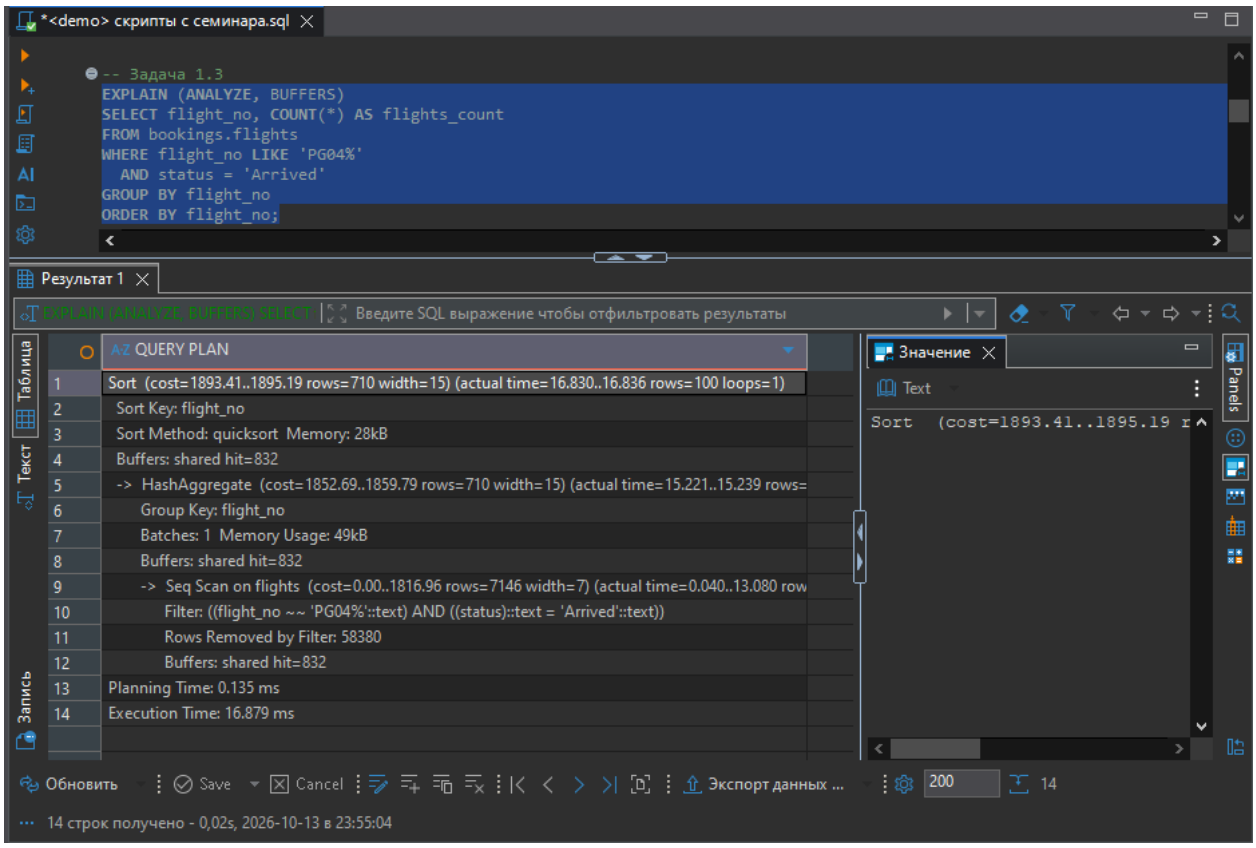
```
--EXPLAIN (ANALYZE, BUFFERS)
SELECT flight_no, COUNT(*) AS flights_count
FROM bookings.flights
WHERE flight_no LIKE 'PG04%'
  AND status = 'Arrived'
GROUP BY flight_no
ORDER BY flight_no;
```

Вывод:

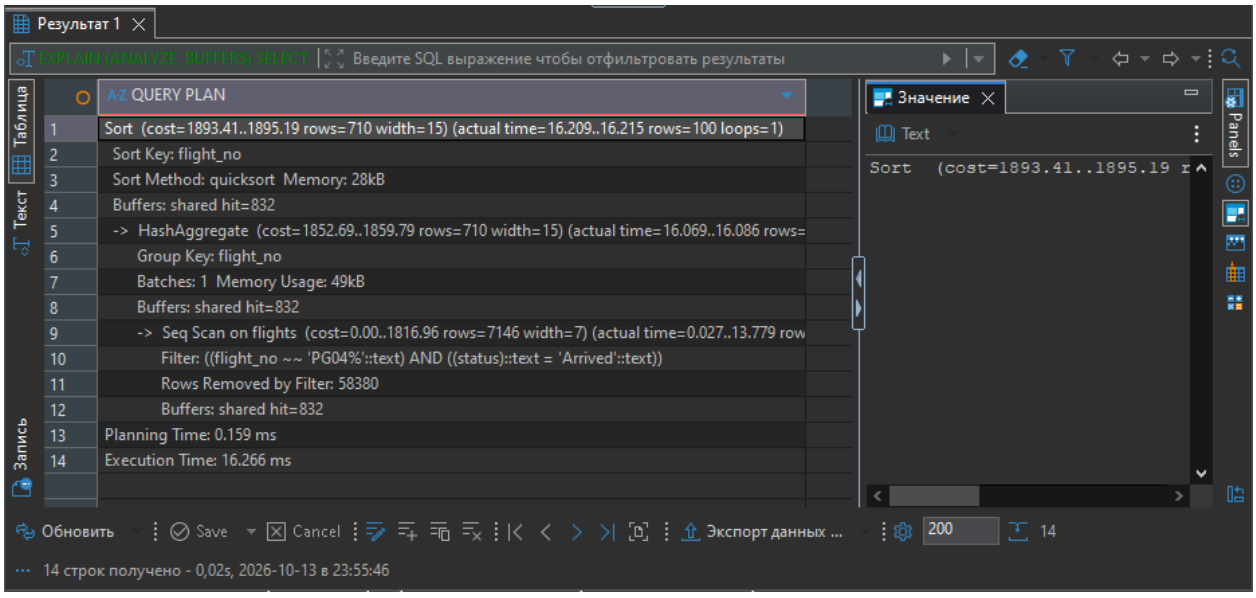


	flight_no	flights_count
1	PG0400	91
2	PG0401	91
3	PG0402	91
4	PG0403	91
5	PG0404	90
6	PG0405	91
7	PG0406	90
8	PG0407	91
9	PG0408	91
10	PG0409	91
11	PG0410	91
12	PG0411	91
13	PG0412	91
14	PG0413	90
15	PG0414	91

1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



Аналитическое резюме:

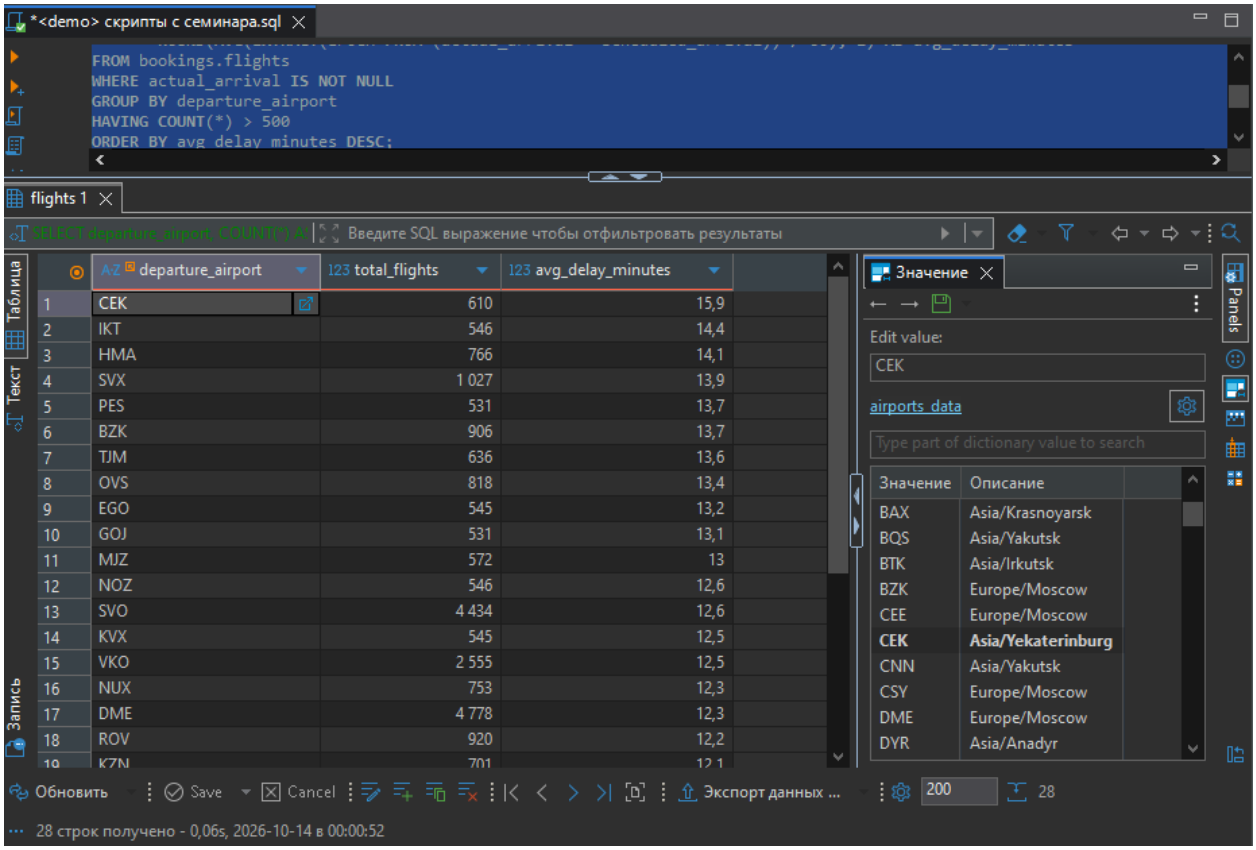
План Seq Scan по flights, агрегация HashAggregate, финальная сортировка quicksort в памяти (28 кБ). Первый запуск hit=832, read=0 (всё из кэша, 16.9 мс), второй hit=832, read=0 (всё из кэша, 16.3 мс). Индекс: по flight_no и status индексов в базе нет, поэтому Seq Scan единственный вариант, хотя префиксный LIKE 'PG04%' и является sargable (в отличие от '%PG04').

Задача 2.1. Для каждого аэропорта вылета: число рейсов и среднее опоздание прибытия в минутах. Оставить только аэропорты с более чем 500 рейсами, отсортировать по убыванию среднего опоздания.

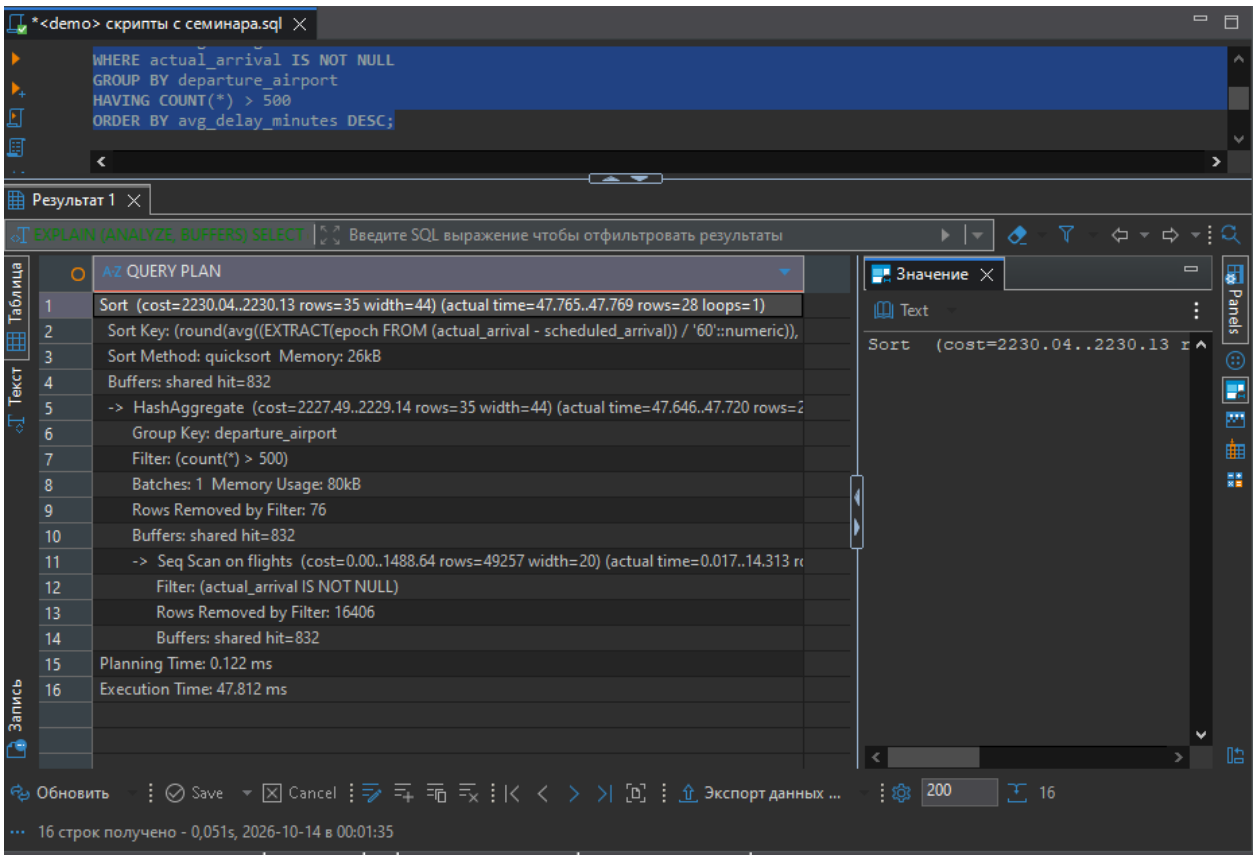
```
EXPLAIN (ANALYZE, BUFFERS)
```

```sql
SELECT departure_airport, COUNT(*) AS total_flights,
    ROUND(AVG(EXTRACT(EPOCH FROM (actual_arrival - scheduled_arrival)) / 60), 1) AS
avg_delay_minutes
FROM bookings.flights
WHERE actual_arrival IS NOT NULL
GROUP BY departure_airport
HAVING COUNT(*) > 500
ORDER BY avg_delay_minutes DESC;
```

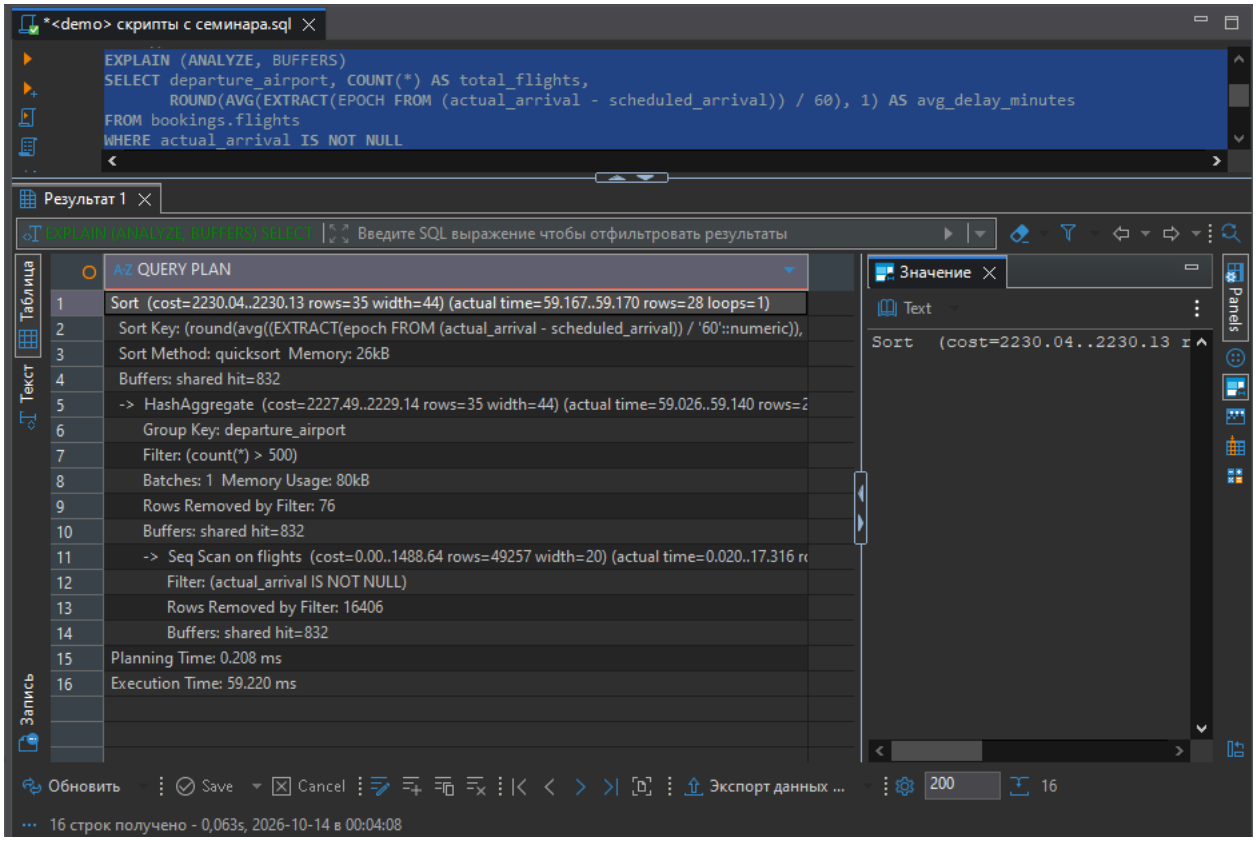
Вывод:



1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



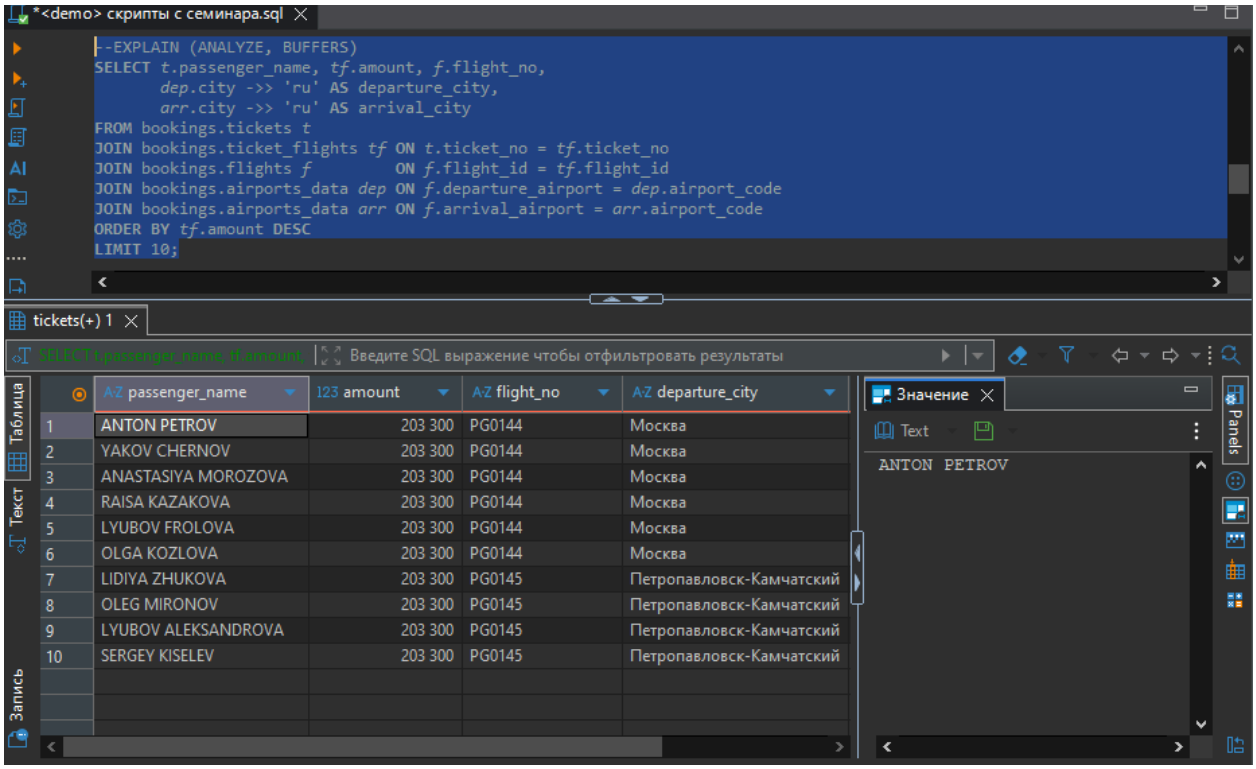
Аналитическое резюме:

План Seq Scan по flights, агрегация HashAggregate с фильтром HAVING COUNT(*) > 500 после группировки, финальная сортировка quicksort в памяти (26 кБ). Первый запуск hit=832, read=0 (всё из кэша, 47.8 мс), второй hit=832, read=0 (всё из кэша, 59.2 мс). Индекс: по departure_airport и actual_arrival индексов нет, поэтому Seq Scan - единственный вариант; HAVING не может использовать индекс, так как фильтрует уже агрегированные группы.

Задача 3.1. Топ-10 самых дорогих проданных билетов: имя пассажира, сумма, номер рейса, город вылета и город прилёта.

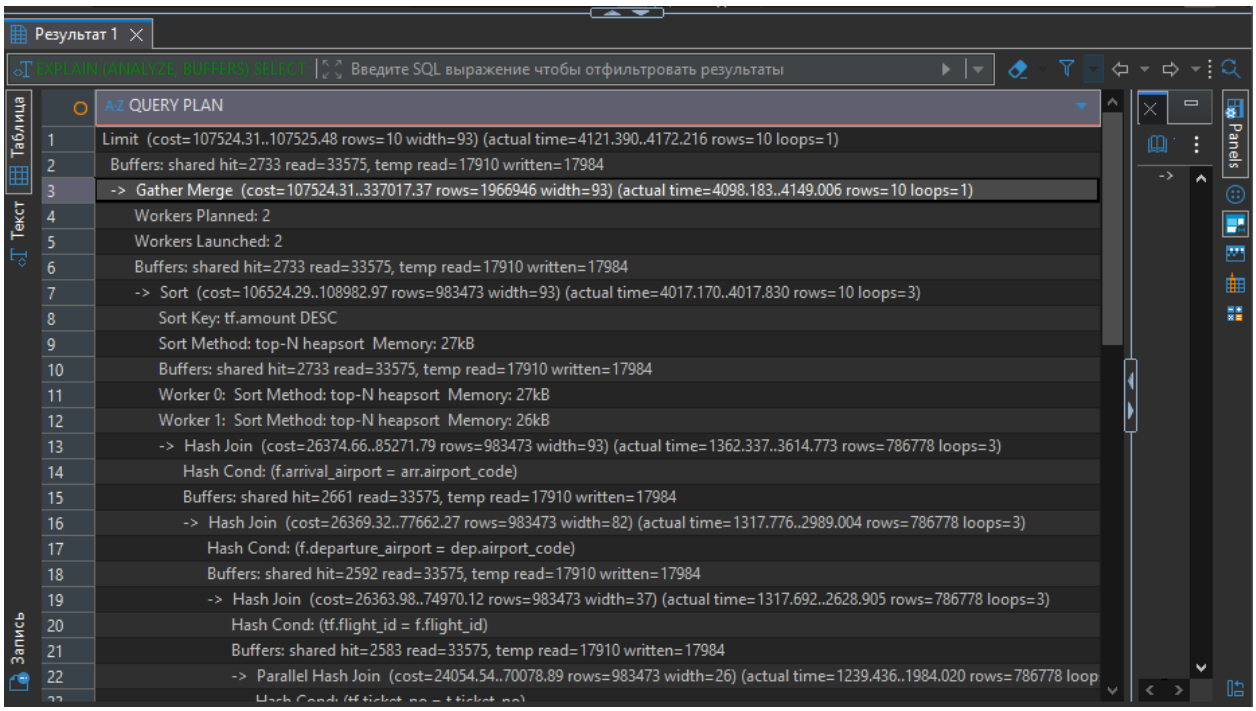
```
--EXPLAIN (ANALYZE, BUFFERS)
SELECT t.passenger_name, tf.amount, f.flight_no,
       dep.city ->> 'ru' AS departure_city,
       arr.city ->> 'ru' AS arrival_city
FROM bookings.tickets t
JOIN bookings.ticket_flights tf ON t.ticket_no = tf.ticket_no
JOIN bookings.flights f         ON f.flight_id = tf.flight_id
JOIN bookings.airports_data dep ON f.departure_airport = dep.airport_code
JOIN bookings.airports_data arr ON f.arrival_airport = arr.airport_code
ORDER BY tf.amount DESC
LIMIT 10;
```

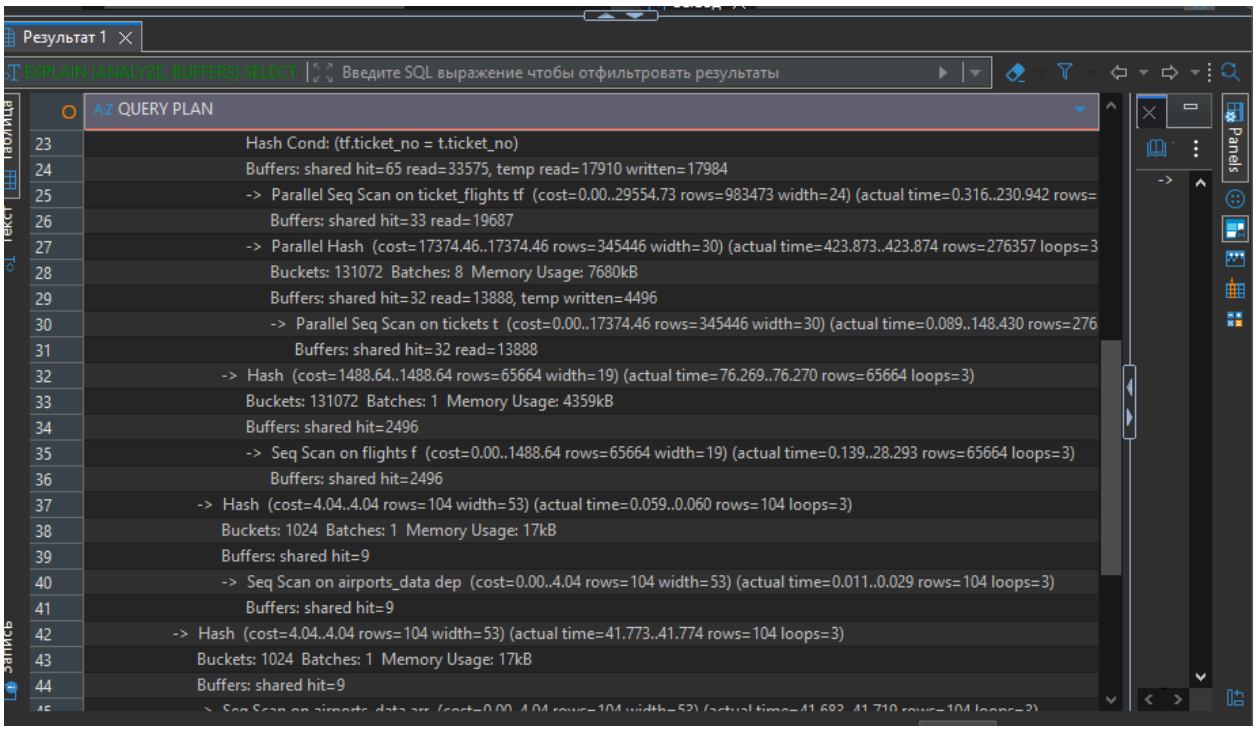
Вывод:

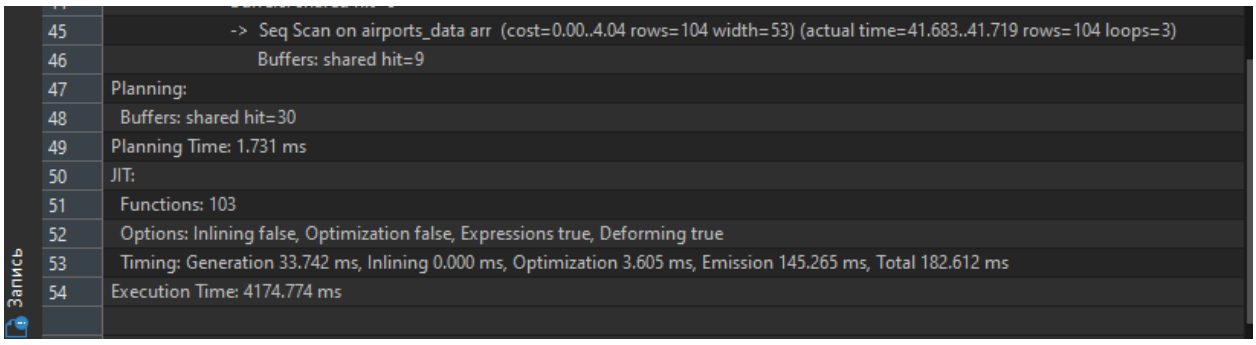


	passenger_name	amount	flight_no	departure_city
1	ANTON PETROV	203 300	PG0144	Москва
2	YAKOV CHERNOV	203 300	PG0144	Москва
3	ANASTASIYA MOROZOVA	203 300	PG0144	Москва
4	RAISA KAZAKOVA	203 300	PG0144	Москва
5	LYUBOV FROLOVA	203 300	PG0144	Москва
6	OLGA KOZLOVA	203 300	PG0144	Москва
7	LIDIYA ZHUKOVA	203 300	PG0145	Петропавловск-Камчатский
8	OLEG MIRONOV	203 300	PG0145	Петропавловск-Камчатский
9	LYUBOV ALEKSANDROVA	203 300	PG0145	Петропавловск-Камчатский
10	SERGEY KISELEV	203 300	PG0145	Петропавловск-Камчатский

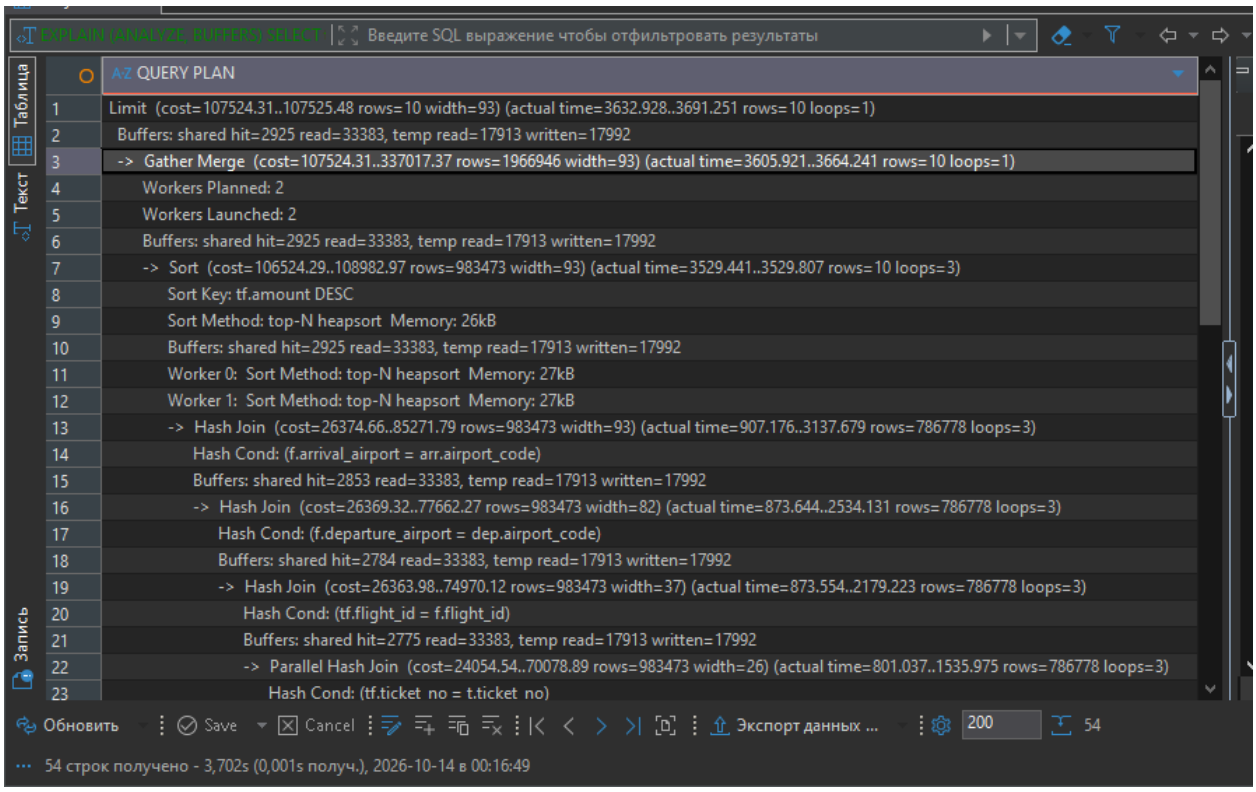
1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):

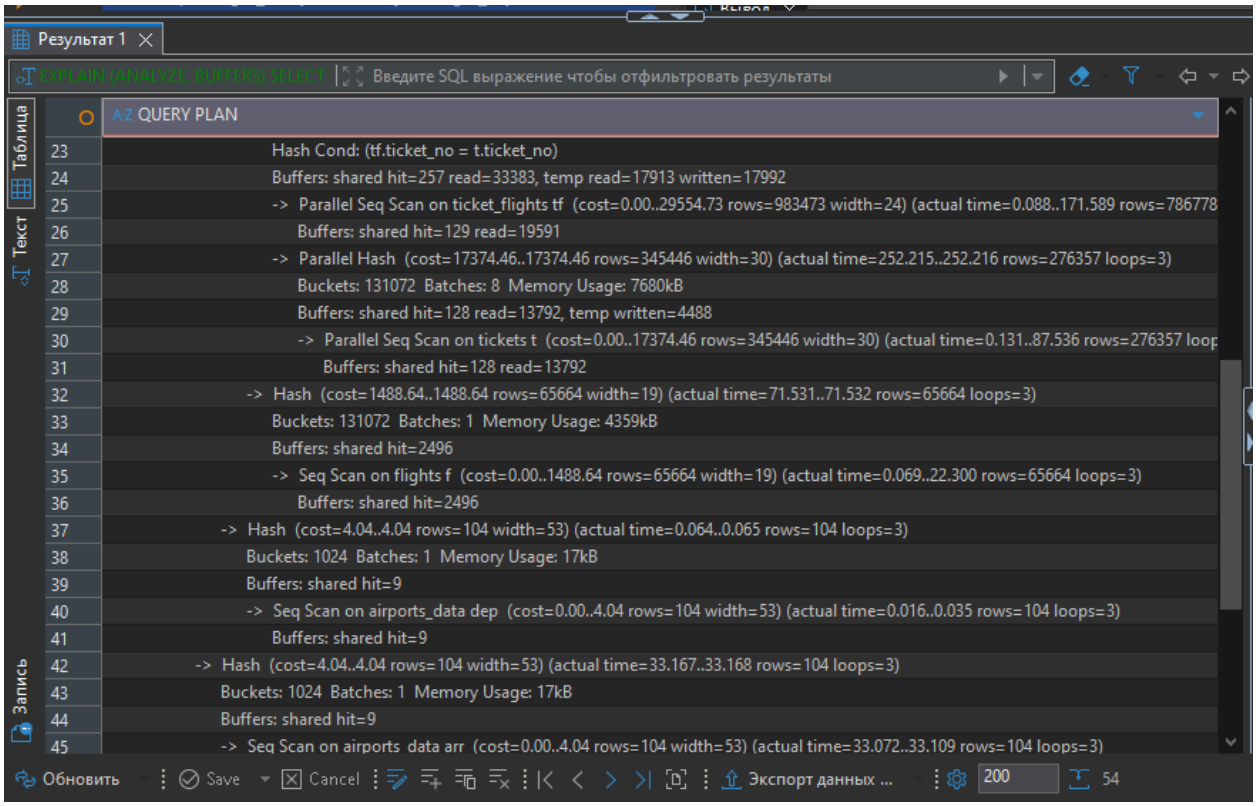


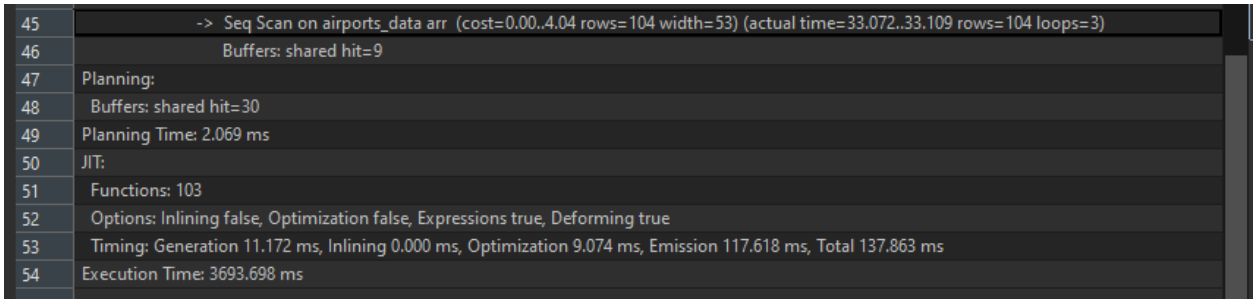




2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):


EXPLAIN (ANALYZE, BUFFERS) SELECT
Введите SQL выражение чтобы отфильтровать результаты
Таблица
Текст
Запись
QUERY PLAN
1 Limit (cost=107524.31..107525.48 rows=10 width=93) (actual time=3632.928..3691.251 rows=10 loops=1)
2 Buffers: shared hit=2925 read=33383, temp read=17913 written=17992
3 -> Gather Merge (cost=107524.31..337017.37 rows=1966946 width=93) (actual time=3605.921..3664.241 rows=10 loops=1)
4 Workers Planned: 2
5 Workers Launched: 2
6 Buffers: shared hit=2925 read=33383, temp read=17913 written=17992
7 -> Sort (cost=106524.29..108982.97 rows=983473 width=93) (actual time=3529.441..3529.807 rows=10 loops=3)
8 Sort Key: tf.amount DESC
9 Sort Method: top-N heapsort Memory: 26kB
10 Buffers: shared hit=2925 read=33383, temp read=17913 written=17992
11 Worker 0: Sort Method: top-N heapsort Memory: 27kB
12 Worker 1: Sort Method: top-N heapsort Memory: 27kB
13 -> Hash Join (cost=26374.66..85271.79 rows=983473 width=93) (actual time=907.176..3137.679 rows=786778 loops=3)
14 Hash Cond: (f.arrival_airport = arr.airport_code)
15 Buffers: shared hit=2853 read=33383, temp read=17913 written=17992
16 -> Hash Join (cost=26369.32..77662.27 rows=983473 width=82) (actual time=873.644..2534.131 rows=786778 loops=3)
17 Hash Cond: (f.departure_airport = dep.airport_code)
18 Buffers: shared hit=2784 read=33383, temp read=17913 written=17992
19 -> Hash Join (cost=26363.98..74970.12 rows=983473 width=37) (actual time=873.554..2179.223 rows=786778 loops=3)
20 Hash Cond: (tf.flight_id = f.flight_id)
21 Buffers: shared hit=2775 read=33383, temp read=17913 written=17992
22 -> Parallel Hash Join (cost=24054.54..70078.89 rows=983473 width=26) (actual time=801.037..1535.975 rows=786778 loops=3)
23 Hash Cond: (tf.ticket_no = t.ticket_no)
Обновить
Save
Cancel
Экспорт данных ...
200
54
54 строк получено - 3,702s (0,001s получ.), 2026-10-14 в 00:16:49


Результат 1
EXPLAIN (ANALYZE, BUFFERS) SELECT
Введите SQL выражение чтобы отфильтровать результаты
Таблица
Текст
Запись
QUERY PLAN
23 Hash Cond: (tf.ticket_no = t.ticket_no)
24 Buffers: shared hit=257 read=33383, temp read=17913 written=17992
25 -> Parallel Seq Scan on ticket_flights tf (cost=0.00..29554.73 rows=983473 width=24) (actual time=0.088..171.589 rows=786778
26 Buffers: shared hit=129 read=19591
27 -> Parallel Hash (cost=17374.46..17374.46 rows=345446 width=30) (actual time=252.215..252.216 rows=276357 loops=3)
28 Buckets: 131072 Batches: 8 Memory Usage: 7680kB
29 Buffers: shared hit=128 read=13792, temp written=4488
30 -> Parallel Seq Scan on tickets t (cost=0.00..17374.46 rows=345446 width=30) (actual time=0.131..87.536 rows=276357 loop
31 Buffers: shared hit=128 read=13792
32 -> Hash (cost=1488.64..1488.64 rows=65664 width=19) (actual time=71.531..71.532 rows=65664 loops=3)
33 Buckets: 131072 Batches: 1 Memory Usage: 4359kB
34 Buffers: shared hit=2496
35 -> Seq Scan on flights f (cost=0.00..1488.64 rows=65664 width=19) (actual time=0.069..22.300 rows=65664 loops=3)
36 Buffers: shared hit=2496
37 -> Hash (cost=4.04..4.04 rows=104 width=53) (actual time=0.064..0.065 rows=104 loops=3)
38 Buckets: 1024 Batches: 1 Memory Usage: 17kB
39 Buffers: shared hit=9
40 -> Seq Scan on airports_data dep (cost=0.00..4.04 rows=104 width=53) (actual time=0.016..0.035 rows=104 loops=3)
41 Buffers: shared hit=9
42 -> Hash (cost=4.04..4.04 rows=104 width=53) (actual time=33.167..33.168 rows=104 loops=3)
43 Buckets: 1024 Batches: 1 Memory Usage: 17kB
44 Buffers: shared hit=9
45 -> Seq Scan on airports_data arr (cost=0.00..4.04 rows=104 width=53) (actual time=33.072..33.109 rows=104 loops=3)
Обновить
Save
Cancel
Экспорт данных ...
200
54


45 -> Seq Scan on airports_data arr (cost=0.00..4.04 rows=104 width=53) (actual time=33.072..33.109 rows=104 loops=3)
46 Buffers: shared hit=9
47 Planning:
48 Buffers: shared hit=30
49 Planning Time: 2.069 ms
50 JIT:
51 Functions: 103
52 Options: Inlining false, Optimization false, Expressions true, Deforming true
53 Timing: Generation 11.172 ms, Inlining 0.000 ms, Optimization 9.074 ms, Emission 117.618 ms, Total 137.863 ms
54 Execution Time: 3693.698 ms

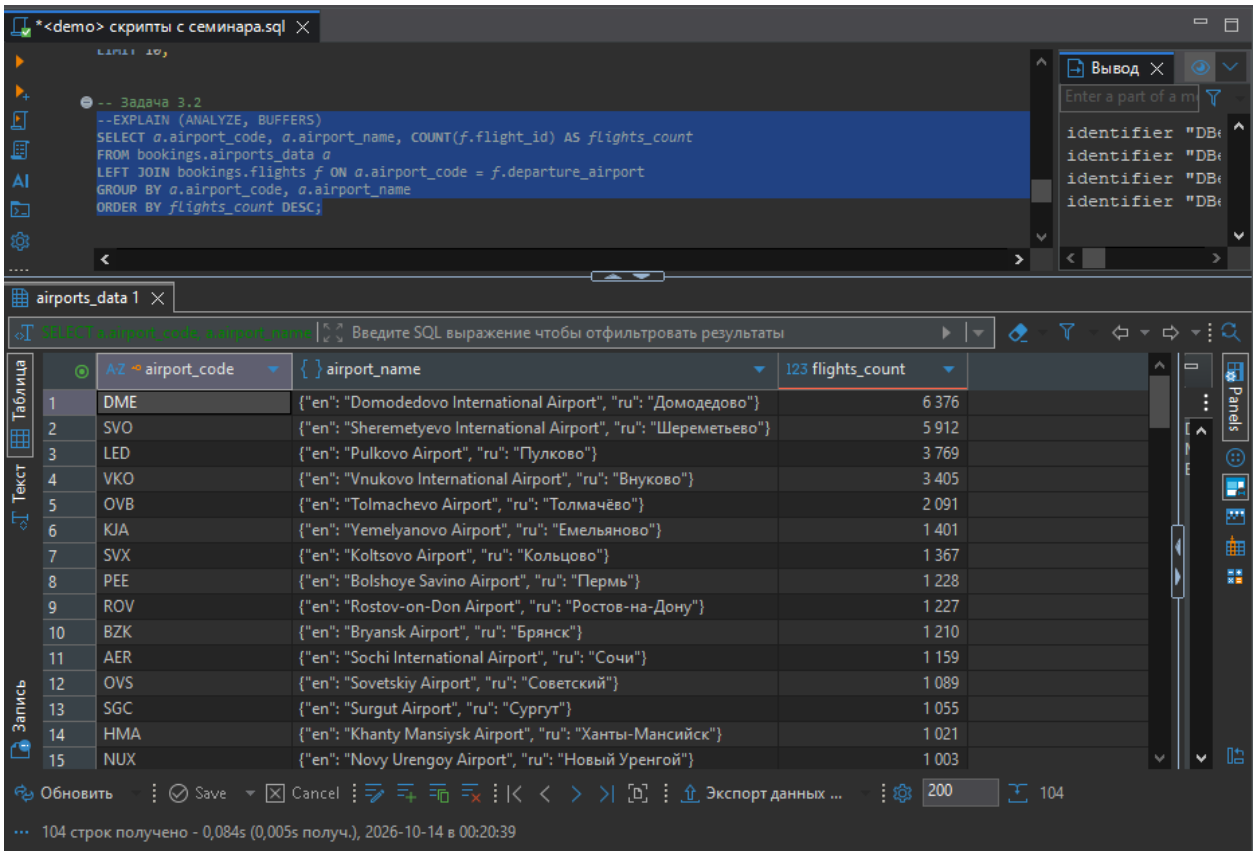
Аналитическое резюме:

План 4 Hash Join, Limit через top-N heapsort (27 кБ). Первый запуск read=33575, hit=2733, temp read=17913 written=17984 (работал с диском и временными файлами, 4174.8 мс), второй запуск read=33383, hit=2925, temp read=17913 written=17992 (снова с диском, 3693.7 мс). Индекс: по tf.amount индекса нет, поэтому сортировка всей выборки из 786 778 строк вынужденная

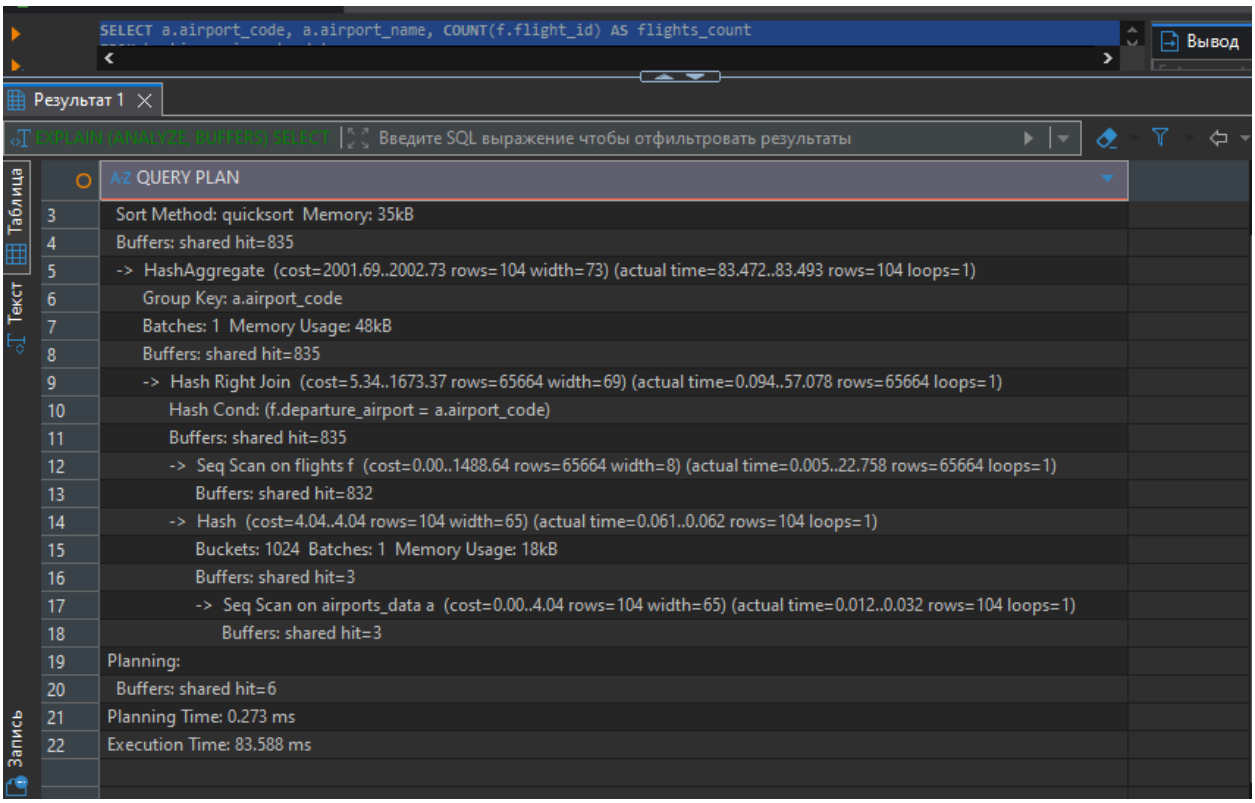
Задача 3.2. Вывести все аэропорты с количеством вылетающих из них рейсов - включая аэропорты, из которых вообще ничего не летает.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT a.airport_code, a.airport_name, COUNT(f.flight_id) AS flights_count
FROM bookings.airports_data a
LEFT JOIN bookings.flights f ON a.airport_code = f.departure_airport
GROUP BY a.airport_code, a.airport_name
ORDER BY flights_count DESC;
```

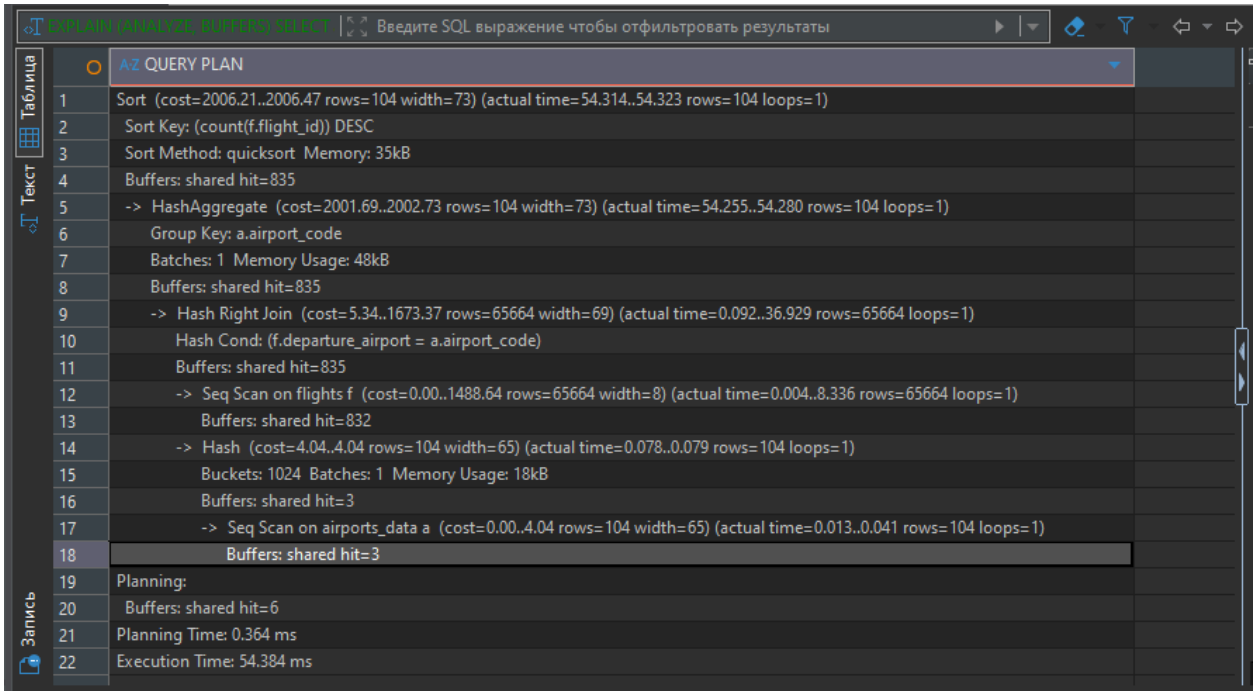
Вывод:



1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



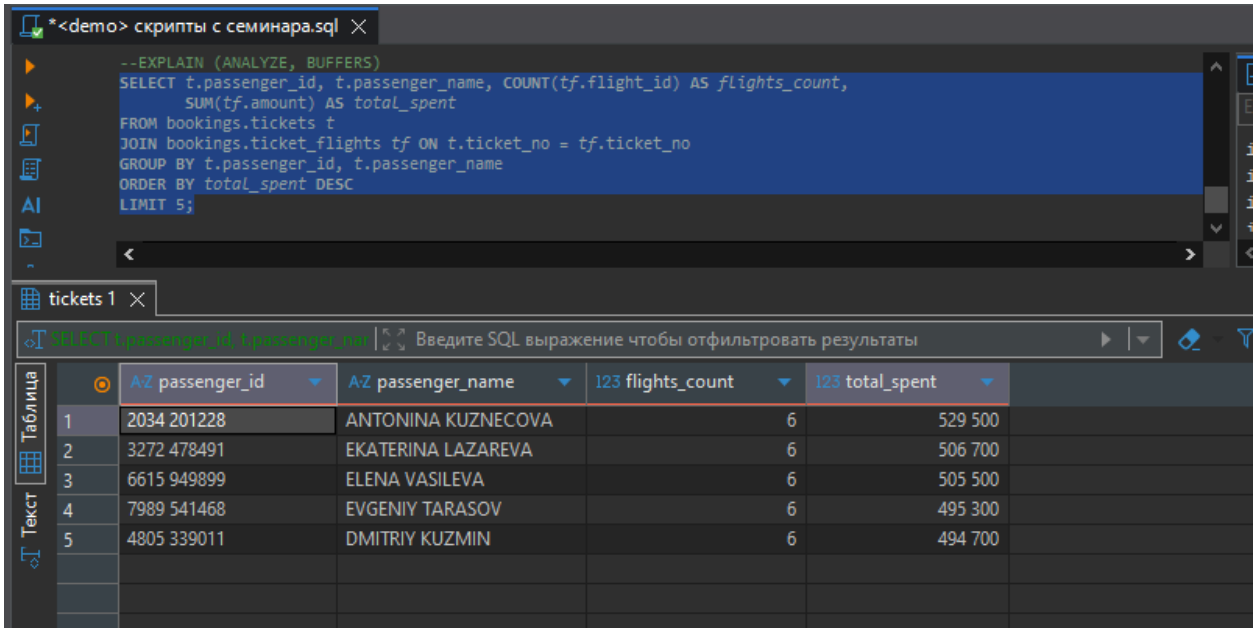
Аналитическое резюме:

План Seq Scan по airports_data + Seq Scan по flights, соединение Hash Right Join (сохраняет все 104 аэропорта), агрегация HashAggregate, сортировка quicksort в памяти (35 кБ). Первый запуск hit=835, read=0 (всё из кэша, 83.6 мс), второй запуск hit=835, read=0 (всё из кэша, 54.4 мс). Индекс: по flights.departure_airport индекса нет, поэтому Seq Scan по 65 664 строкам; COUNT(f.flight_id) вместо COUNT(*) даёт 0 для аэропортов без вылетов.

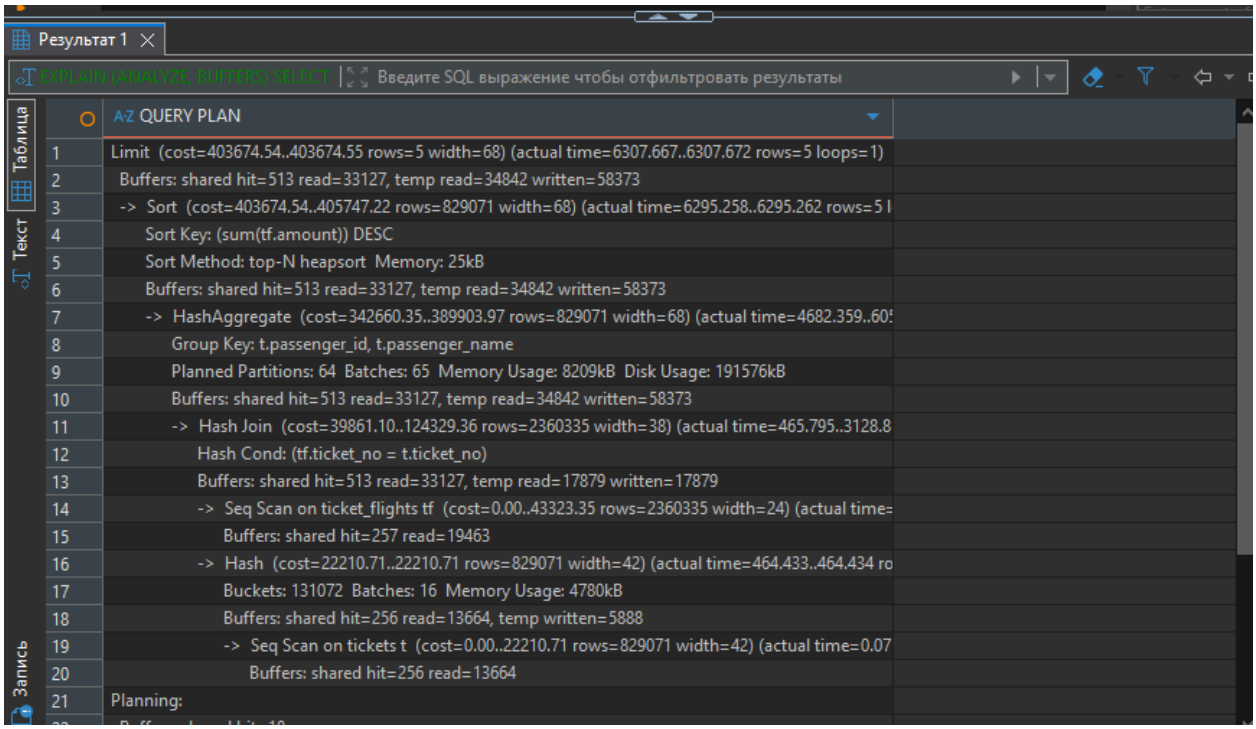
Задача 3.3. Топ-5 пассажиров по суммарным тратам на билеты: id пассажира, имя, число перелётов, суммарная стоимость.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT t.passenger_id, t.passenger_name, COUNT(tf.flight_id) AS flights_count,
    SUM(tf.amount) AS total_spent
FROM bookings.tickets t
JOIN bookings.ticket_flights tf ON t.ticket_no = tf.ticket_no
GROUP BY t.passenger_id, t.passenger_name
ORDER BY total_spent DESC
LIMIT 5;
```

Вывод:

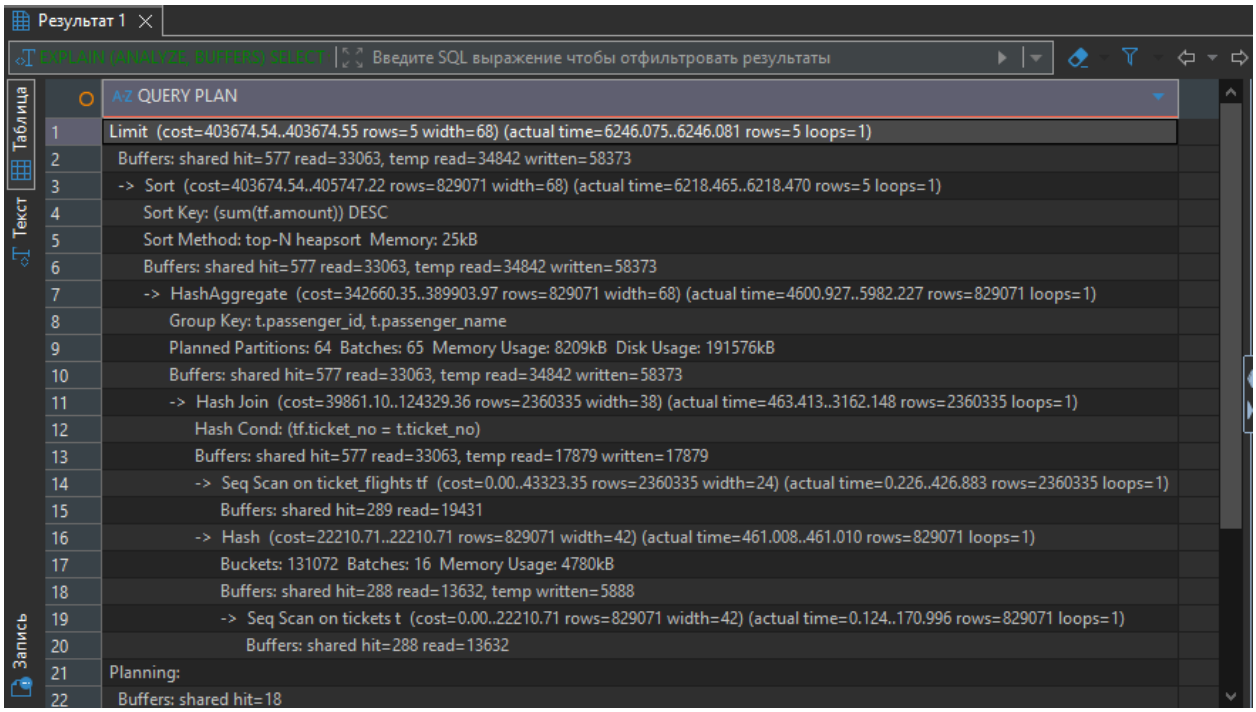


1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



```
                    Buffers: shared hit=256 read=13664
Planning:
  Buffers: shared hit=18
Planning Time: 0.460 ms
JIT:
  Functions: 18
  Options: Inlining false, Optimization false, Expressions true, Deforming true
  Timing: Generation 1.746 ms, Inlining 0.000 ms, Optimization 0.894 ms, Emission 17.153 ms, Total 19.7
Execution Time: 6354.014 ms
```

2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):




```
  Buffers: shared hit=18
Planning Time: 0.410 ms
JIT:
  Functions: 18
  Options: Inlining false, Optimization false, Expressions true, Deforming true
  Timing: Generation 1.506 ms, Inlining 0.000 ms, Optimization 0.923 ms, Emission 32.244 ms, Total 34.672 ms
Execution Time: 6291.087 ms
```

Аналитическое резюме:

План Hash Join tickets и ticket_flights по ticket_no, HashAggregate с группировкой по passenger_id и passenger_name (64 партиции, 809 МБ памяти, disk usage 191576 кБ), финально Limit через top-N heapsort (25 кБ). Первый запуск read=33127, hit=513, temp read=34842 written=58373 (работал с диском и временными файлами, 6354.0 мс), второй запуск read=33063, hit=577, temp read=34842 written=58373 (снова с диском, 6291.1 мс). Индекс: по t.passenger_id и tf.amount индексов нет, поэтому Seq Scan по tickets (42 млн строк) и ticket_flights (245 МБ), сортировка всей выборки вынужденная.

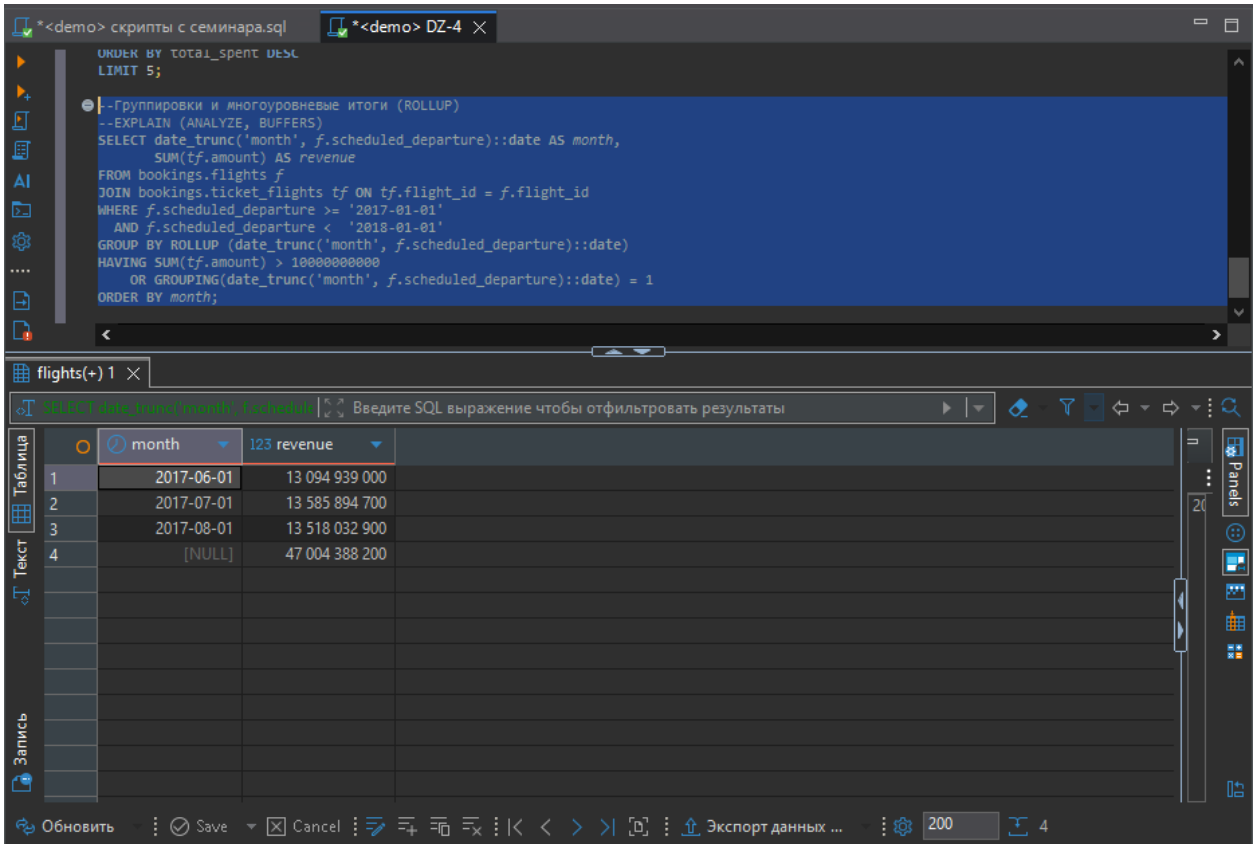
Группировки и многоуровневые итоги (ROLLUP): Посчитайте помесячную выручку от продажи билетов за 2017 год вместе с общим итогом - одним запросом, через ROLLUP.

Оставьте только месяцы с выручкой более 10 млрд рублей (HAVING), следя за тем, чтобы итоговая строка при этом не потерялась.

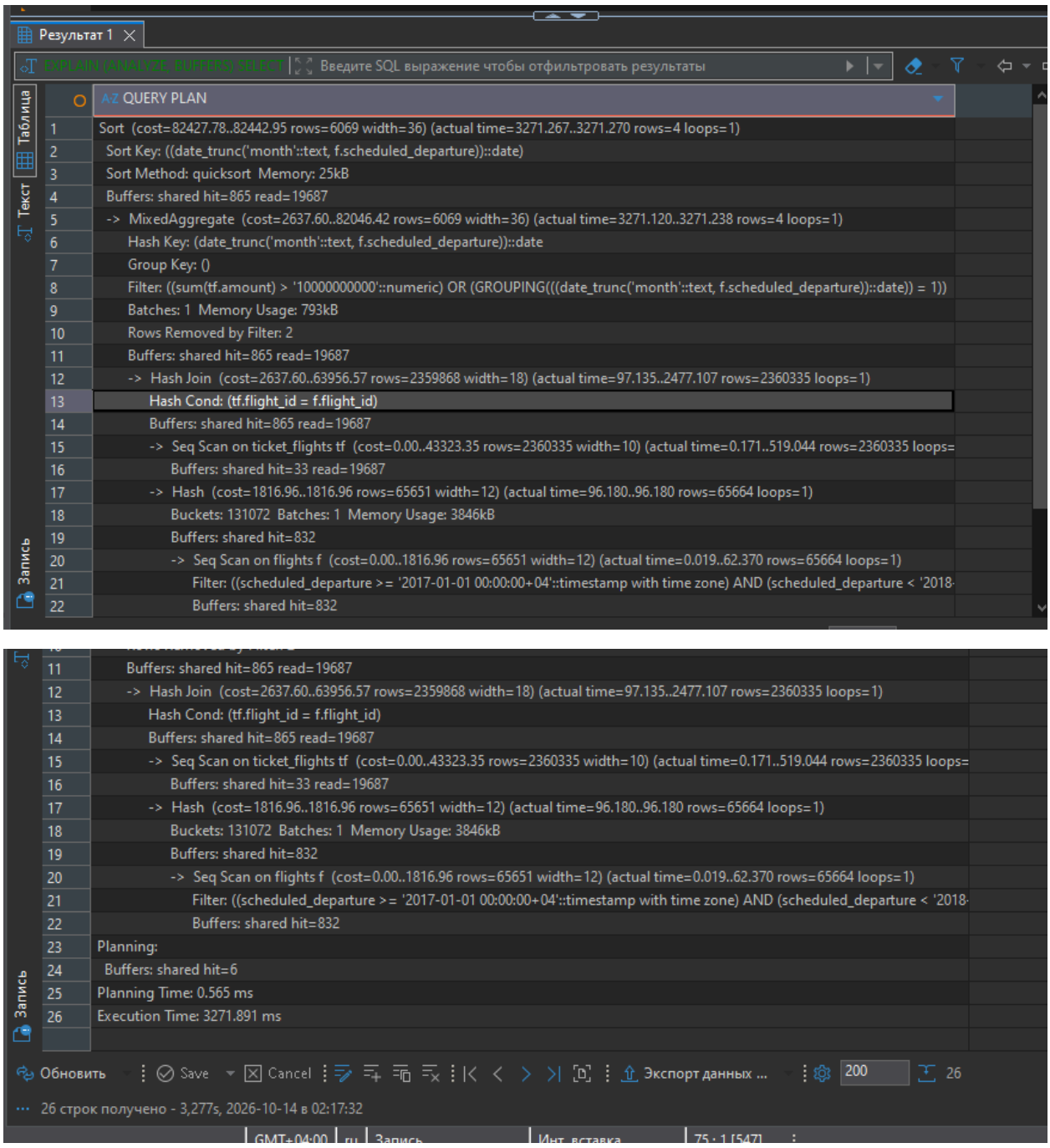
Приложите EXPLAIN ANALYZE. Объясните, как HAVING работает вместе с ROLLUP и почему итоговая строка (со значением NULL в колонке месяца) проходит фильтр.

```
--Группировки и многоуровневые итоги (ROLLUP)
--EXPLAIN (ANALYZE, BUFFERS)
SELECT date_trunc('month', f.scheduled_departure)::date AS month,
    SUM(tf.amount) AS revenue
FROM bookings.flights f
JOIN bookings.ticket_flights tf ON tf.flight_id = f.flight_id
WHERE f.scheduled_departure >= '2017-01-01'
  AND f.scheduled_departure <  '2018-01-01'
GROUP BY ROLLUP (date_trunc('month', f.scheduled_departure)::date)
HAVING SUM(tf.amount) > 10000000000
   OR GROUPING(date_trunc('month', f.scheduled_departure)::date) = 1
ORDER BY month;
```

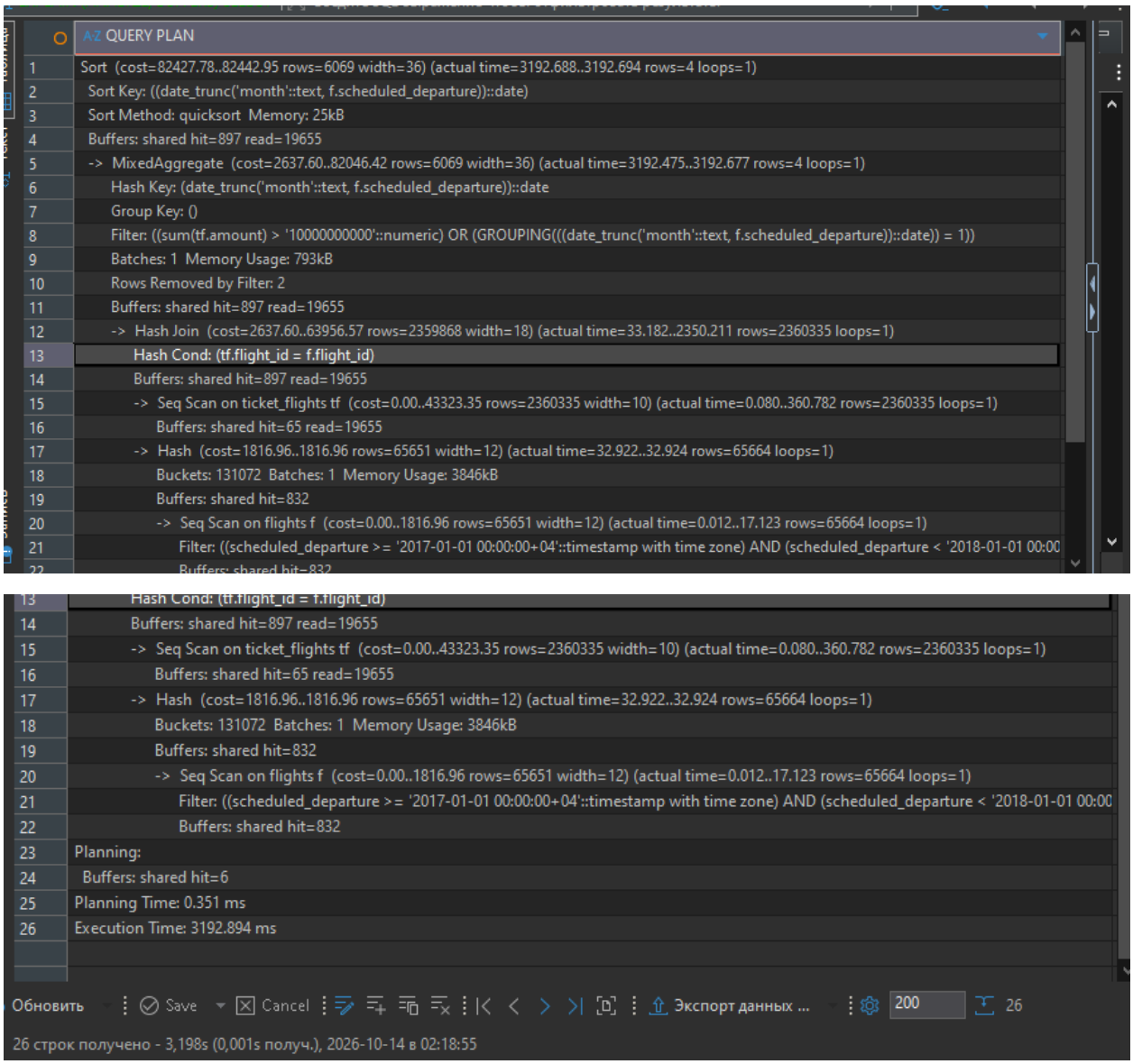
Вывод:



1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



Аналитическое резюме:

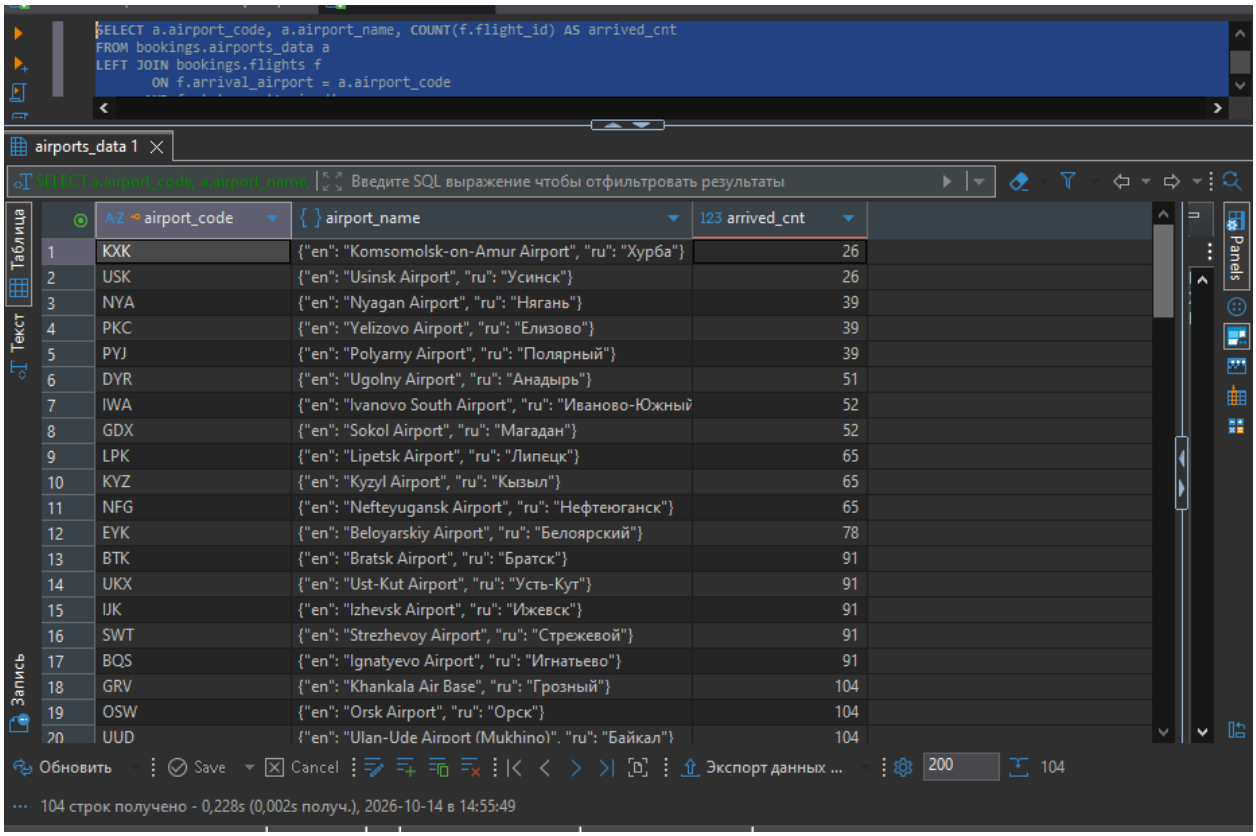
План Seq Scan по flights и ticket_flights, Hash Join по flight_id, MixedAggregate с группировкой ROLLUP по date_trunc(month) (помесячные группы + итоговая), HAVING с фильтром sum(amount) > 10 млрд ИЛИ GROUPING(...) = 1, финальная сортировка quicksort в памяти (25 кБ). Первый запуск hit=865, read=19687 (часть с диска, 3271.3 мс), второй hit=897, read=19655 (снова с диском, 3192.9 мс). Индекс: по scheduled_departure и amount индексов нет, Seq Scan по 65 664 и 2 360 335 строкам - единственный вариант. HAVING применяется после ROLLUP; итоговая строка (month = NULL) проходит фильтр за счёт GROUPING(...) = 1, а не за счёт проверки на NULL.

LEFT JOIN и условная агрегация: Для каждого аэропорта прибытия посчитайте число прибывших рейсов (status = 'Arrived'), включая аэропорты, куда не прибыл ни один рейс: код аэропорта, название, число рейсов.

```
SELECT a.airport_code, a.airport_name, COUNT(f.flight_id) AS arrived_cnt
FROM bookings.airports_data a
LEFT JOIN bookings.flights f
    ON f.arrival_airport = a.airport_code
   AND f.status = 'Arrived'
GROUP BY a.airport_code, a.airport_name
```

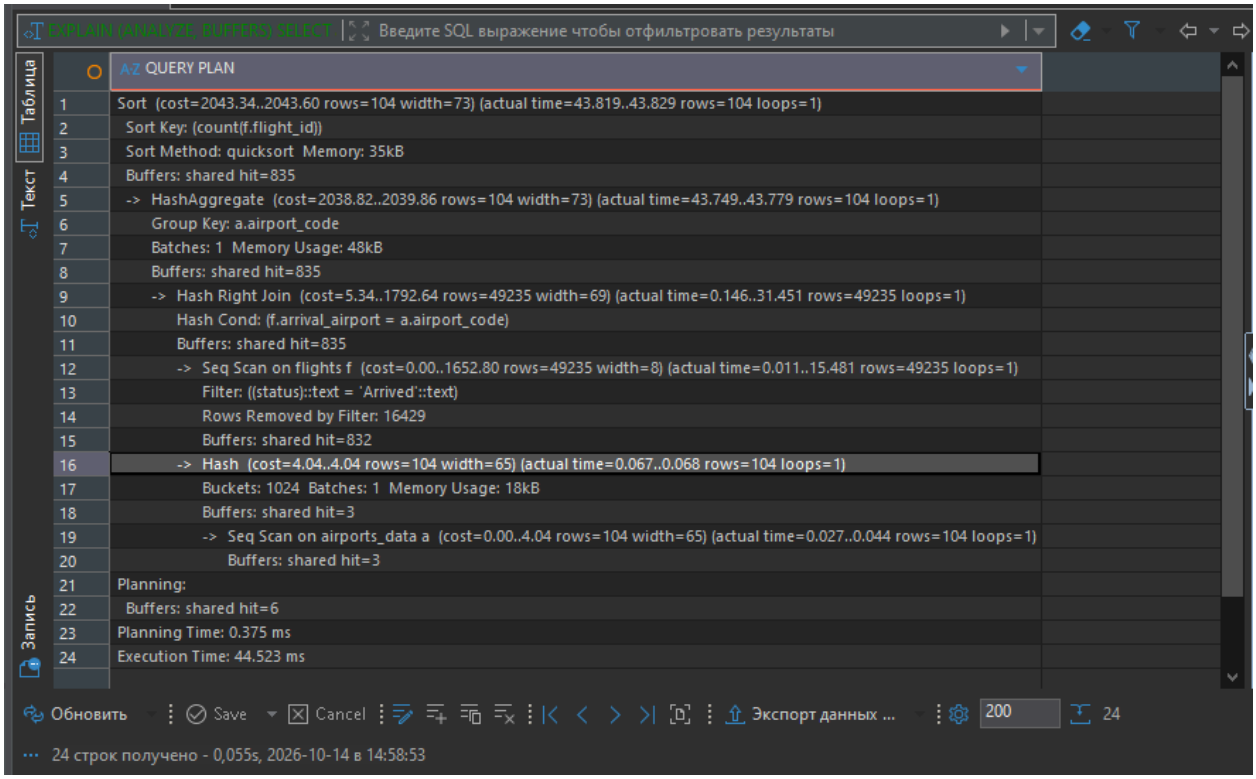
```
ORDER BY arrived_cnt;
```

Вывод:

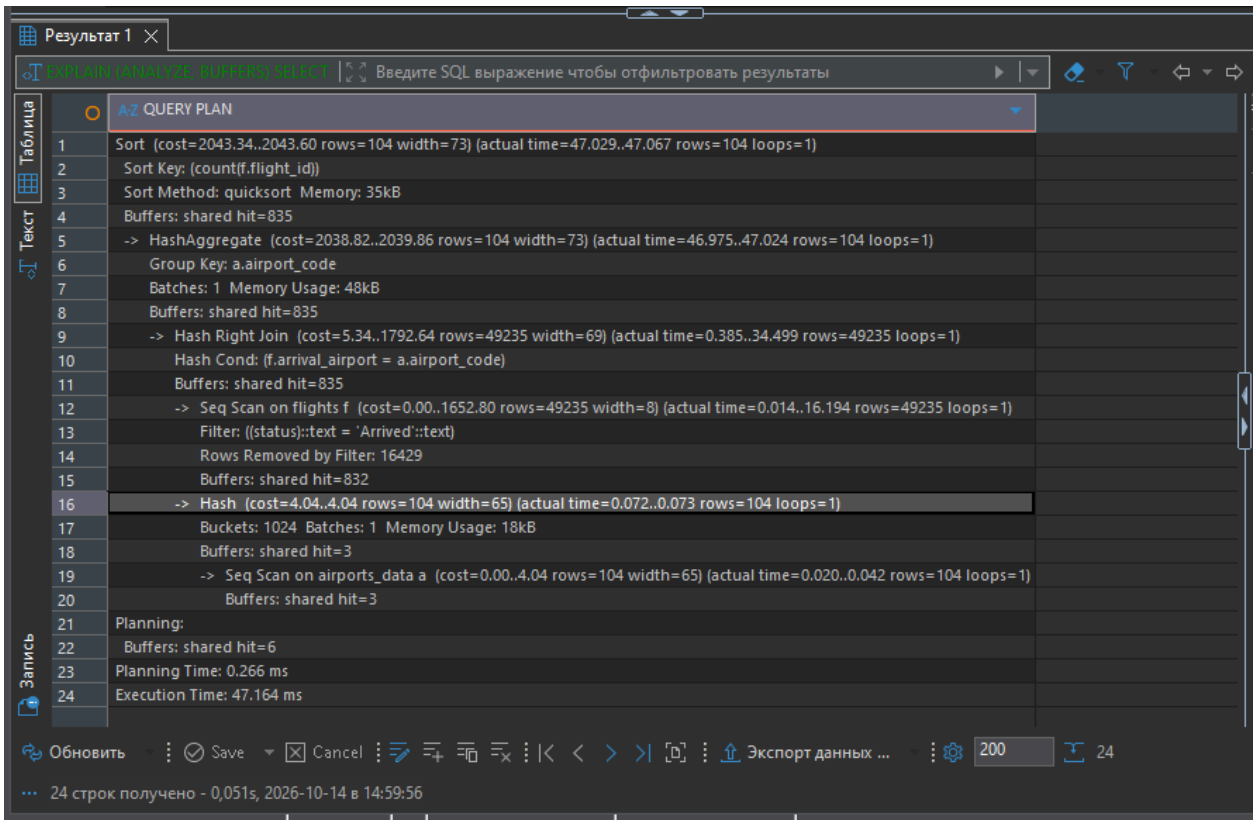


	airport_code	airport_name	arrived_cnt
1	KXK	{"en": "Komsomolsk-on-Amur Airport", "ru": "Хурба"}	26
2	USK	{"en": "Usinsk Airport", "ru": "Усинск"}	26
3	NYA	{"en": "Nyagan Airport", "ru": "Нягань"}	39
4	PKC	{"en": "Yelizovo Airport", "ru": "Елизово"}	39
5	PYJ	{"en": "Polyarny Airport", "ru": "Полярный"}	39
6	DYR	{"en": "Ugolny Airport", "ru": "Анадырь"}	51
7	IWA	{"en": "Ivanovo South Airport", "ru": "Иваново-Южный	52
8	GDX	{"en": "Sokol Airport", "ru": "Магадан"}	52
9	LPK	{"en": "Lipetsk Airport", "ru": "Липецк"}	65
10	KYZ	{"en": "Kyzyl Airport", "ru": "Кызыл"}	65
11	NFG	{"en": "Nefteyugansk Airport", "ru": "Нефтеюганск"}	65
12	EYK	{"en": "Beloyarskiy Airport", "ru": "Белоярский"}	78
13	BTK	{"en": "Bratsk Airport", "ru": "Братск"}	91
14	UKX	{"en": "Ust-Kut Airport", "ru": "Усть-Кут"}	91
15	IJK	{"en": "Izhevsk Airport", "ru": "Ижевск"}	91
16	SWT	{"en": "Strezhevoy Airport", "ru": "Стрежевой"}	91
17	BQS	{"en": "Ignatyevo Airport", "ru": "Игнатьево"}	91
18	GRV	{"en": "Khankala Air Base", "ru": "Грозный"}	104
19	OSW	{"en": "Orsk Airport", "ru": "Орск"}	104
20	UUD	{"en": "Ulan-Ude Airport (Mukhino)", "ru": "Байкал"}	104

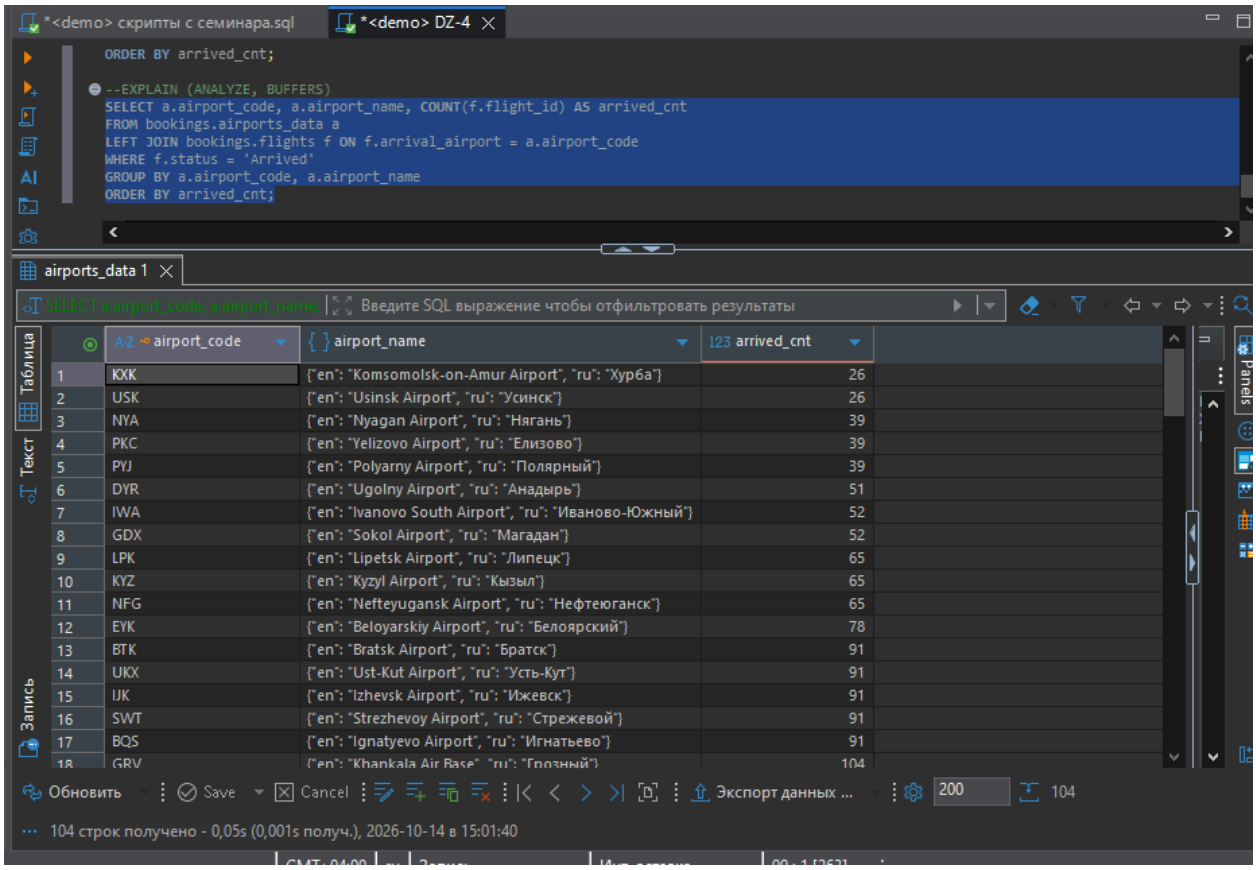
1 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS):



Контрольный запуск:



Ответы на вопросы:

1. **Почему здесь недопустим COUNT(*):**

После LEFT JOIN для аэропорта без прибывших рейсов строка всё равно остаётся (с NULL в колонках flights). COUNT(*) считает строки, а не значения - вернёт 1 вместо 0.
Нужен COUNT(f.flight_id) -он игнорирует NULL.

2. Что означает узел Hash Right Join в плане.

Планировщик поменял местами левую и правую таблицы, чтобы меньшую airports_data построить как хэш в памяти (18 кБ), а большую flights сканировать. Логика LEFT JOIN при этом сохраняется - все 104 аэропорта остаются в результате.

Аналитическое резюме:

План - Seq Scan по airports_data и flights, Hash Right Join по arrival_airport = airport_code (сохраняет все 104 аэропорта), HashAggregate, сортировка quicksort в памяти (35 кБ). Первый запуск hit=835, read=0 (всё из кэша, 44.5 мс), второй hit=835, read=0 (всё из кэша, 47.1 мс). Индекс: по flights.arrival_airport индекса нет, Seq Scan - единственный вариант. Контрольный запуск с WHERE вернул те же 104 строки, потому что в базе у всех аэропортов есть хотя бы один прибывший рейс; на данных с «пустым» аэропортом LEFT JOIN деградировал бы до INNER JOIN.

Sargability (Оптимизация предикатов): Найдите все рейсы, выполненные (status = 'Arrived') в выходные дни (суббота и воскресенье) в августе 2017 года.

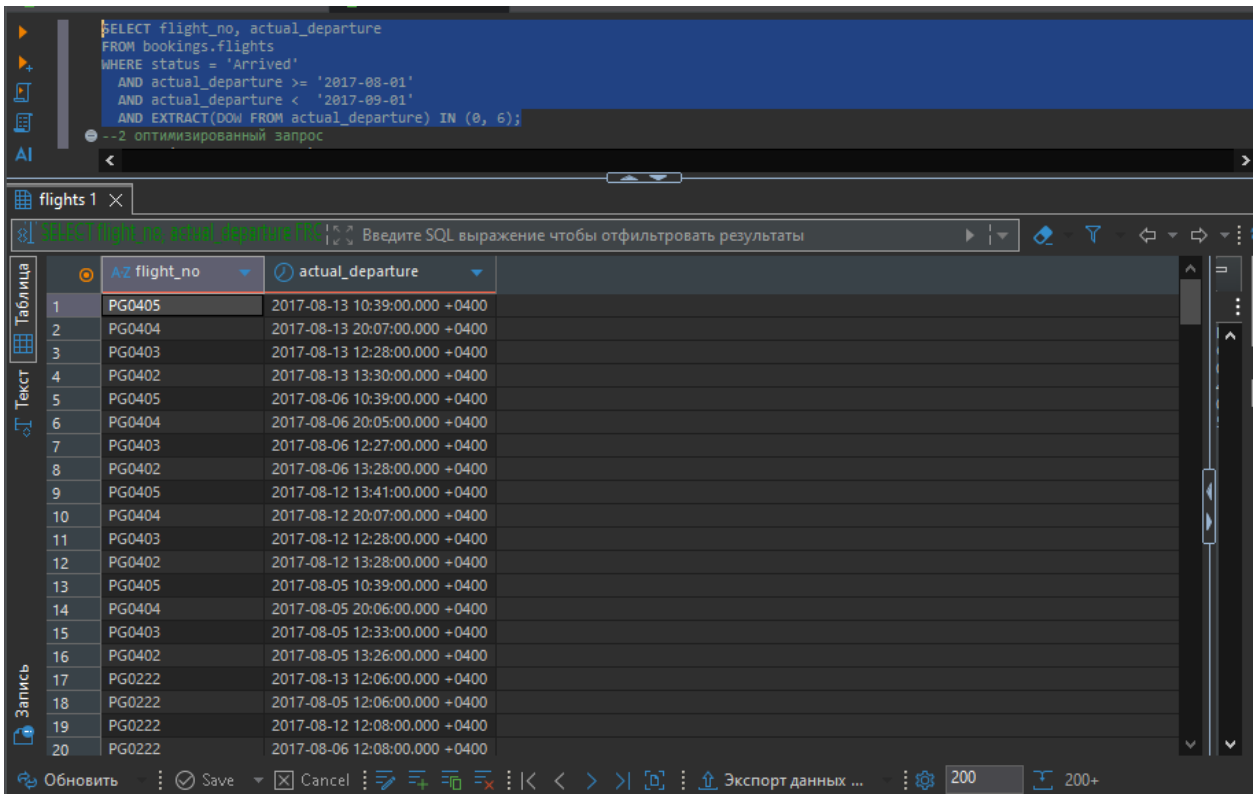
неоптимизированный запрос

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT flight_no, actual_departure
FROM bookings.flights
WHERE status = 'Arrived'
  AND actual_departure >= '2017-08-01'
  AND actual_departure <  '2017-09-01'
  AND EXTRACT(DOW FROM actual_departure) IN (0, 6);
```

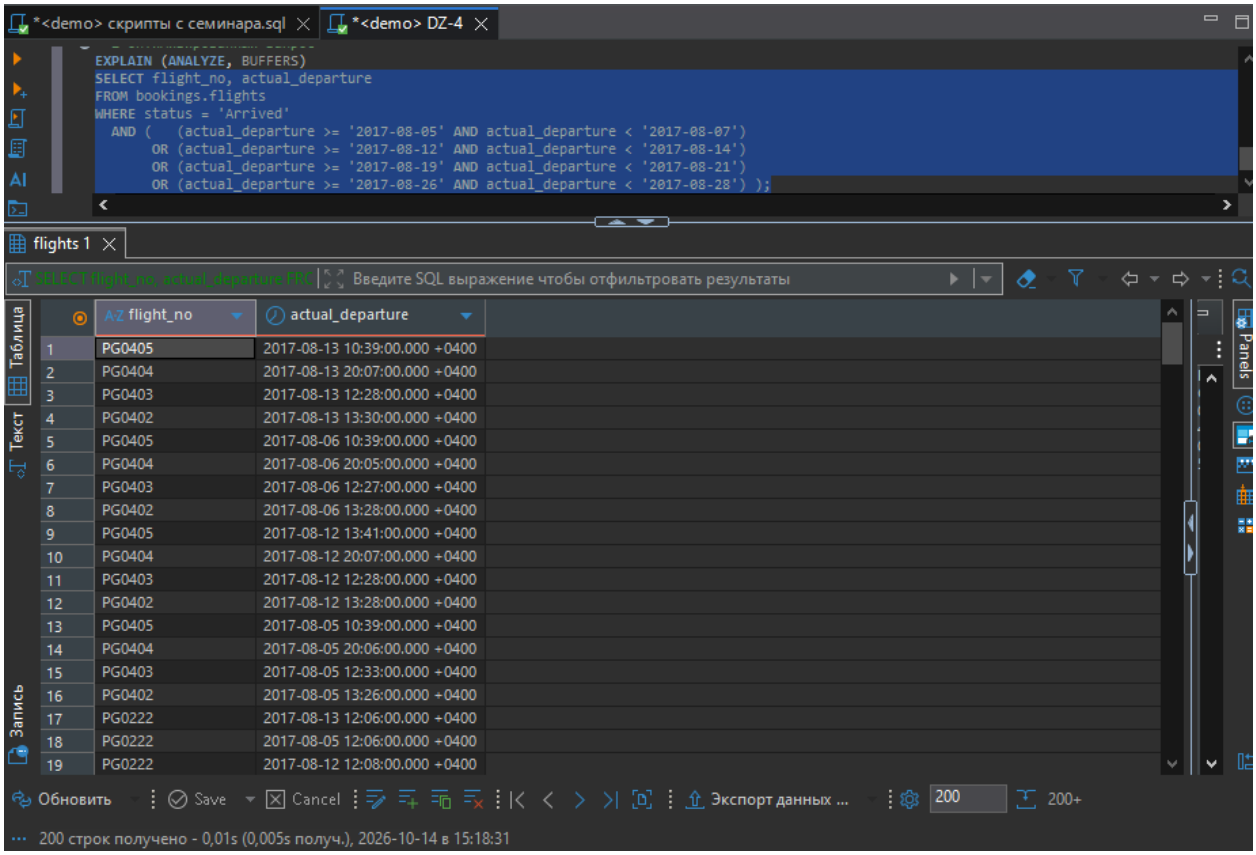
оптимизированный запрос

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT flight_no, actual_departure
FROM bookings.flights
WHERE status = 'Arrived'
  AND (   (actual_departure >= '2017-08-05' AND actual_departure < '2017-08-07')
      OR (actual_departure >= '2017-08-12' AND actual_departure < '2017-08-14')
      OR (actual_departure >= '2017-08-19' AND actual_departure < '2017-08-21')
      OR (actual_departure >= '2017-08-26' AND actual_departure < '2017-08-28') );
```

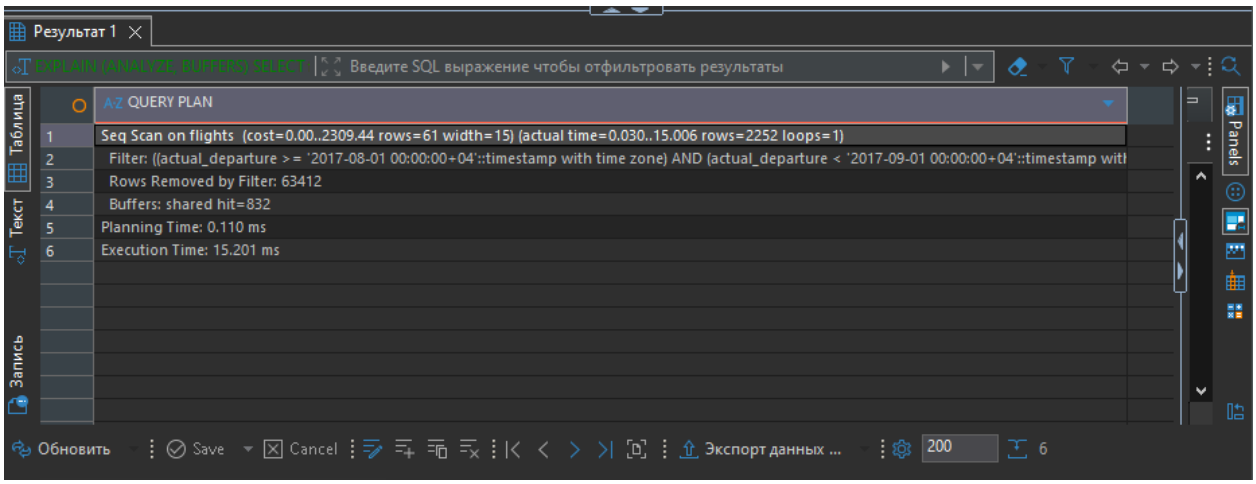
Вывод неоптимизированный запрос**:**



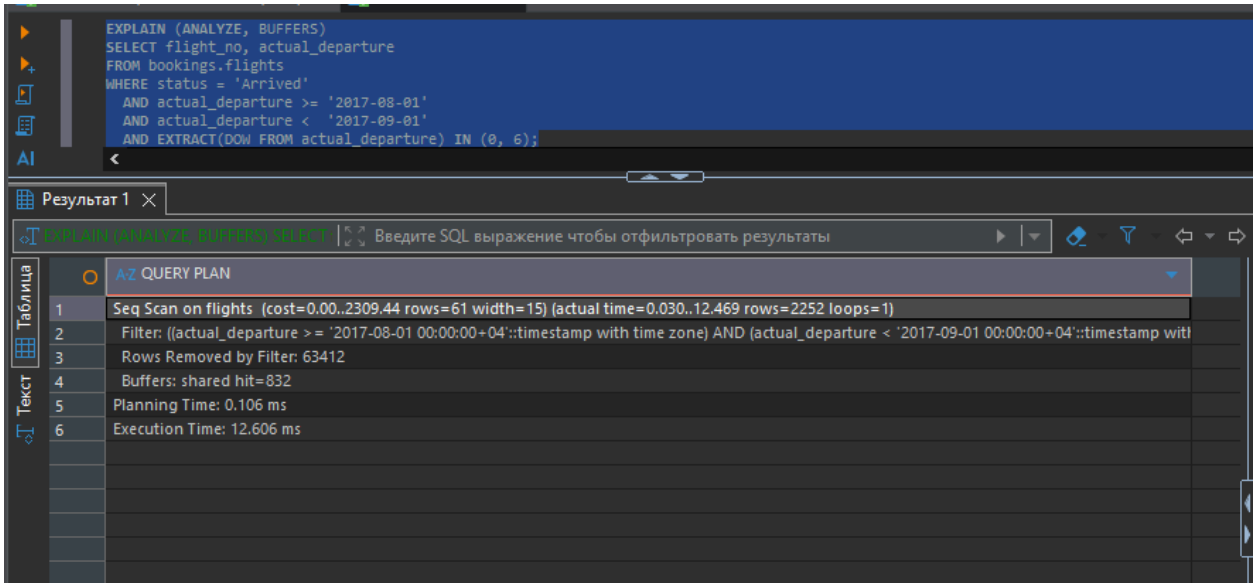
Вывод оптимизированный запрос**:**



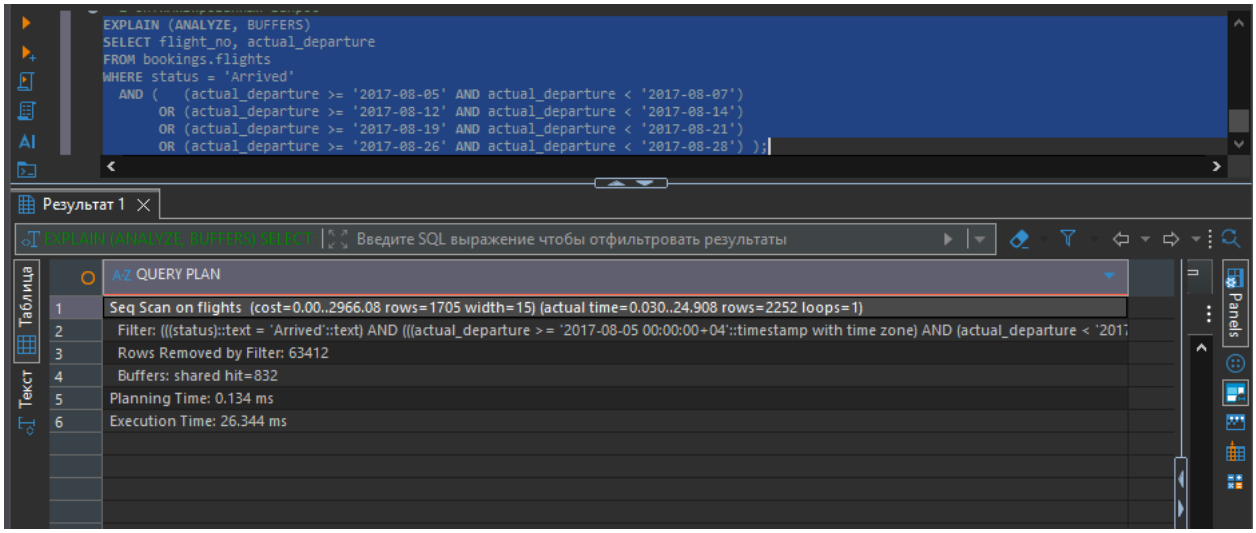
1 Вывод команды EXPLAIN (ANALYZE, BUFFERS) неоптимизированный запрос**:**



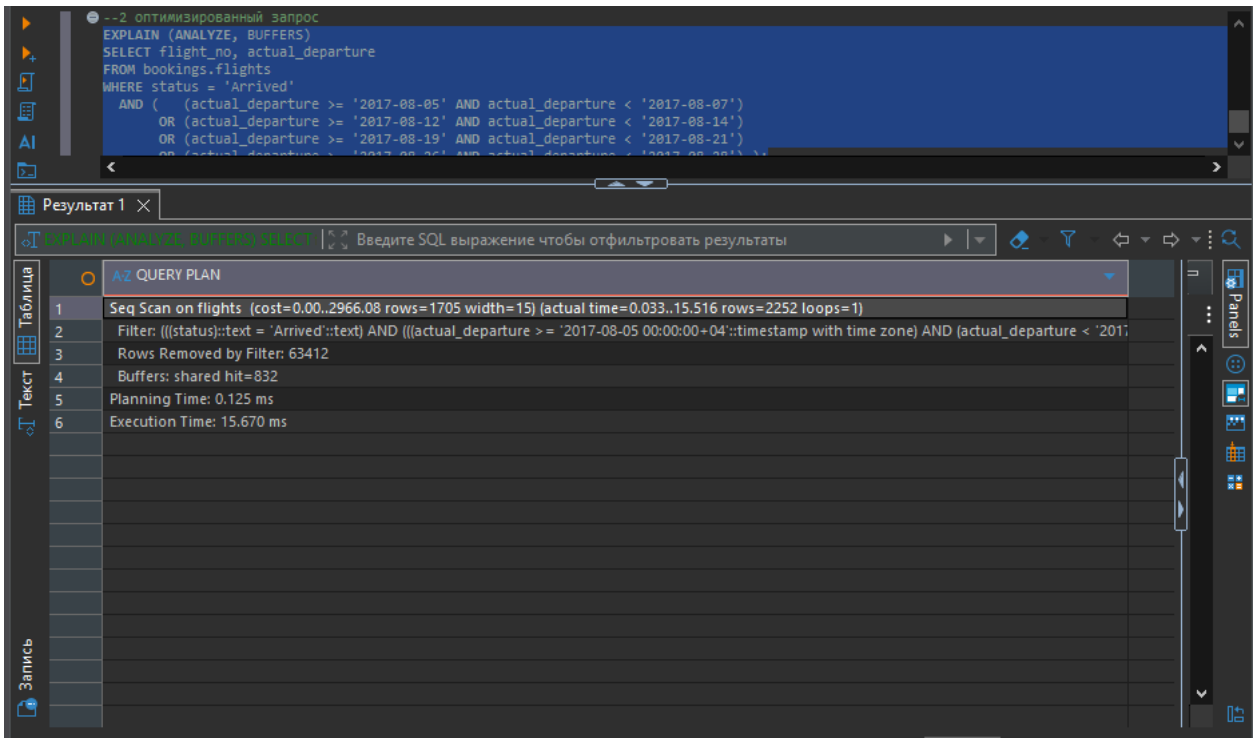
2 Вывод команды EXPLAIN (ANALYZE, BUFFERS) неоптимизированный запрос**:**



1 Вывод команды EXPLAIN (ANALYZE, BUFFERS) оптимизированный запрос**:**



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS) оптимизированный запрос**:**



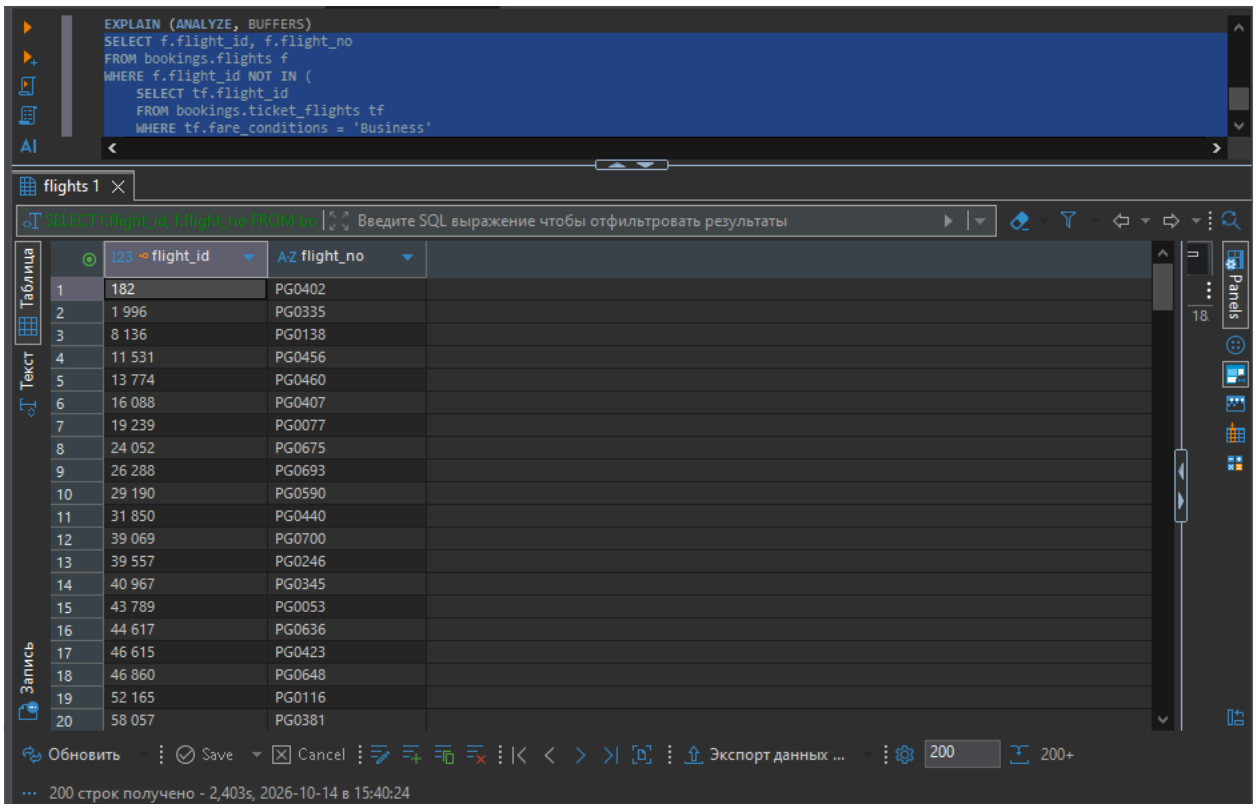
Аналитическое резюме:

Оба запроса дали одинаковый план Seq Scan по flights - индекса по actual_departure в базе нет, поэтому sargability не дал разницы. Неоптимизированный (EXTRACT DOW): hit=832, read=0 (всё из кэша, 12.6 мс), оценка плана rows=61 при факте 2252, отброшено 63 412 строк. Оптимизированный (диапазоны): hit=832, read=0 (всё из кэша, 15.7 мс), оценка rows=1705 при факте 2252, отброшено 63 412 строк. Разница только в оценке селективности: EXTRACT(DOW) планировщик оценивает точнее, диапазоны - занижает. Sargability дал бы эффект при наличии индекса по actual_departure: тогда оптимизированный запрос пошёл бы через Index Scan, а неоптимизированный - всё равно через Seq Scan.

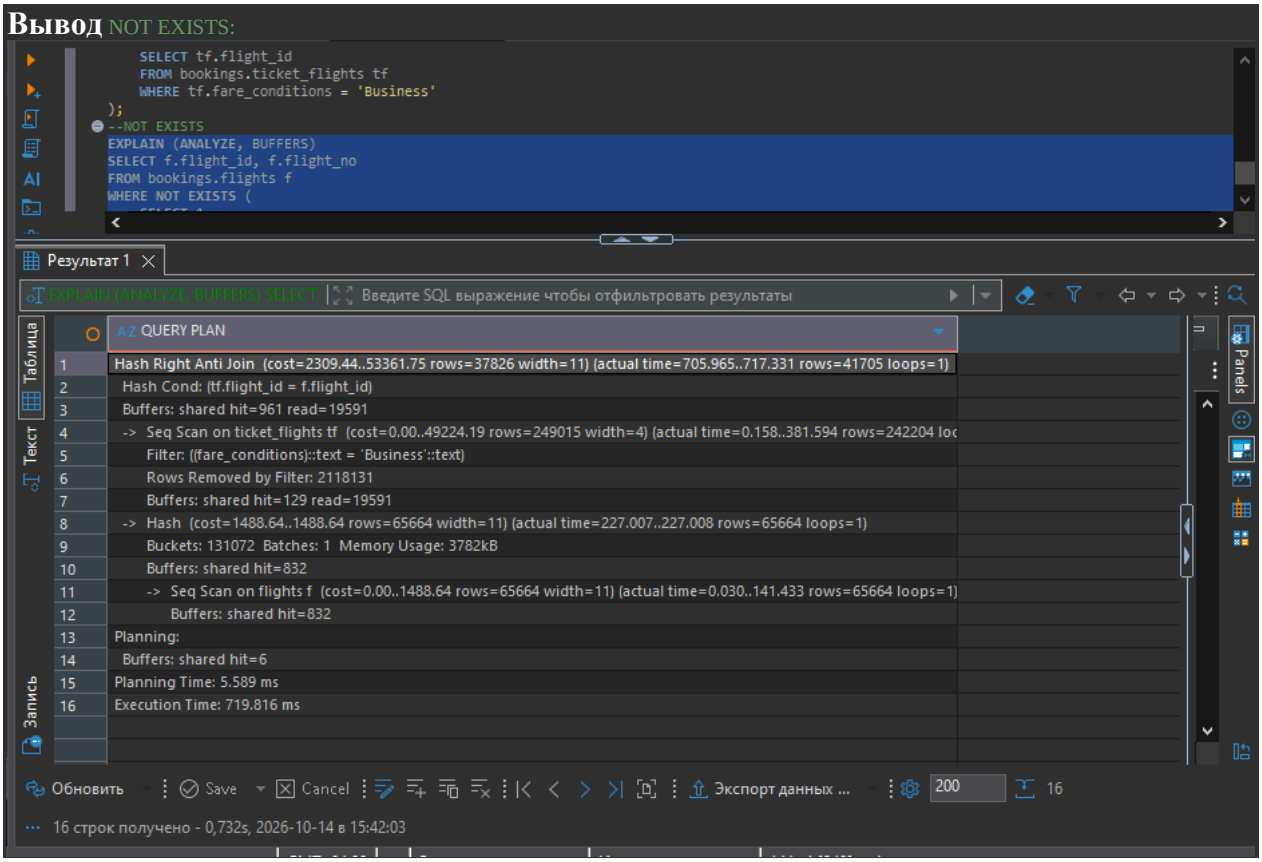
ANTI-JOIN (Оптимизация отрицания): Найдите все рейсы, на которые не был продан ни один билет бизнес-класса (fare_conditions = 'Business').

```
--NOT IN
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no
FROM bookings.flights f
WHERE f.flight_id NOT IN (
   SELECT tf.flight_id
   FROM bookings.ticket_flights tf
   WHERE tf.fare_conditions = 'Business'
);
--NOT EXISTS
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no
FROM bookings.flights f
WHERE NOT EXISTS (
   SELECT 1
   FROM bookings.ticket_flights tf
   WHERE tf.flight_id = f.flight_id
    AND tf.fare_conditions = 'Business'
);
```

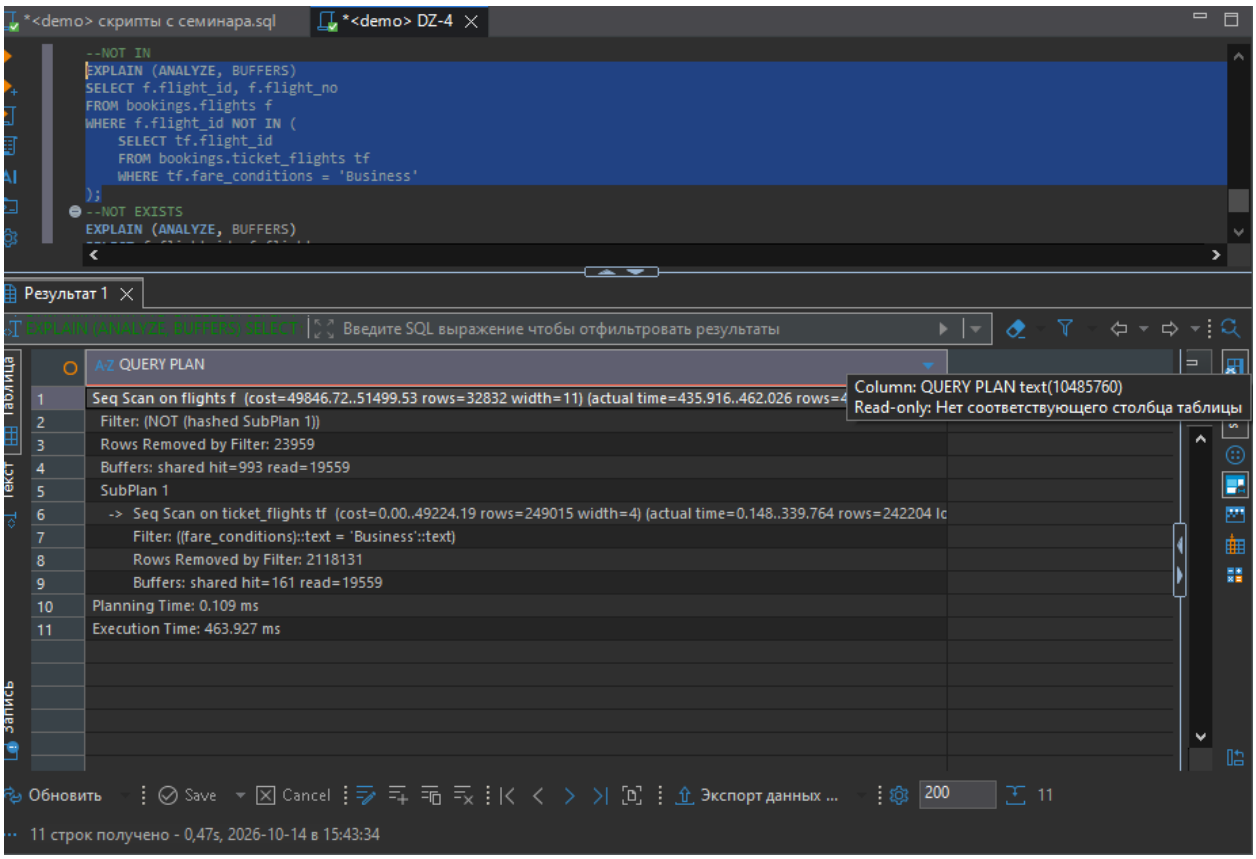
Вывод --NOT IN**:**



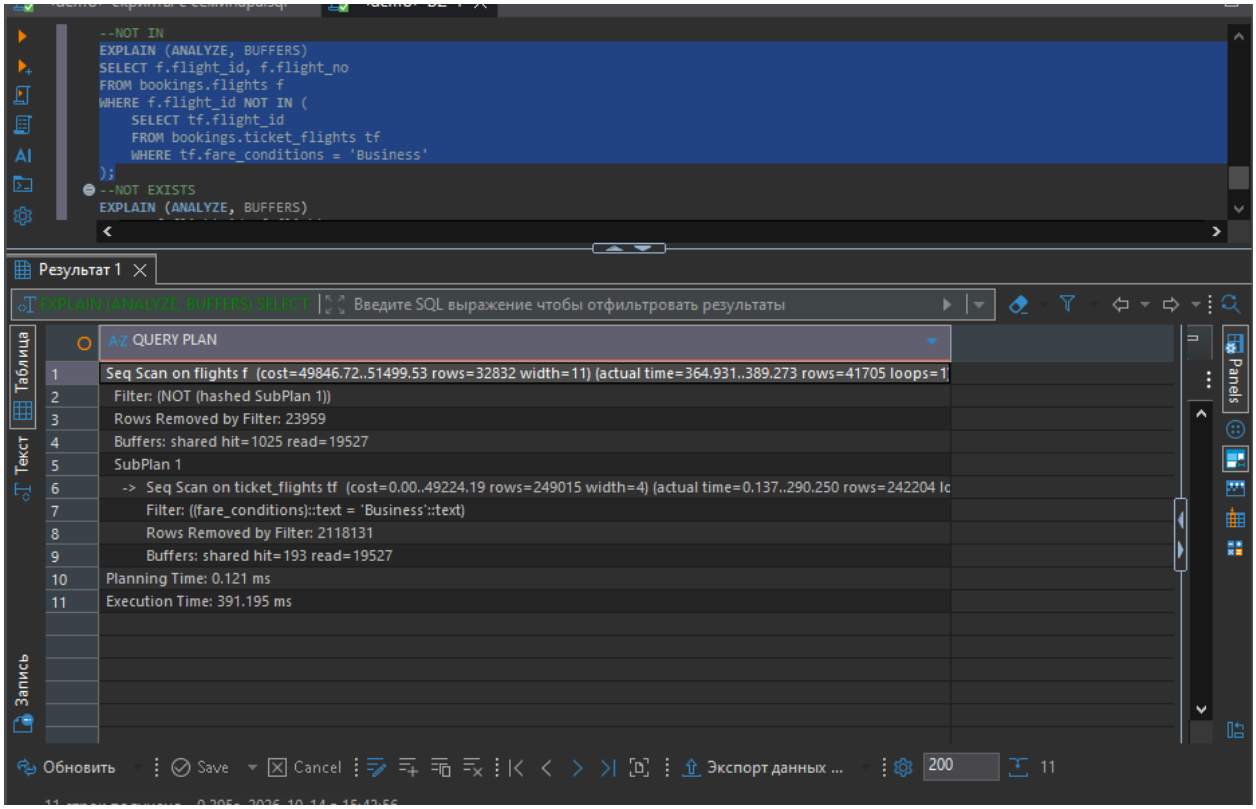
Вывод NOT EXISTS:



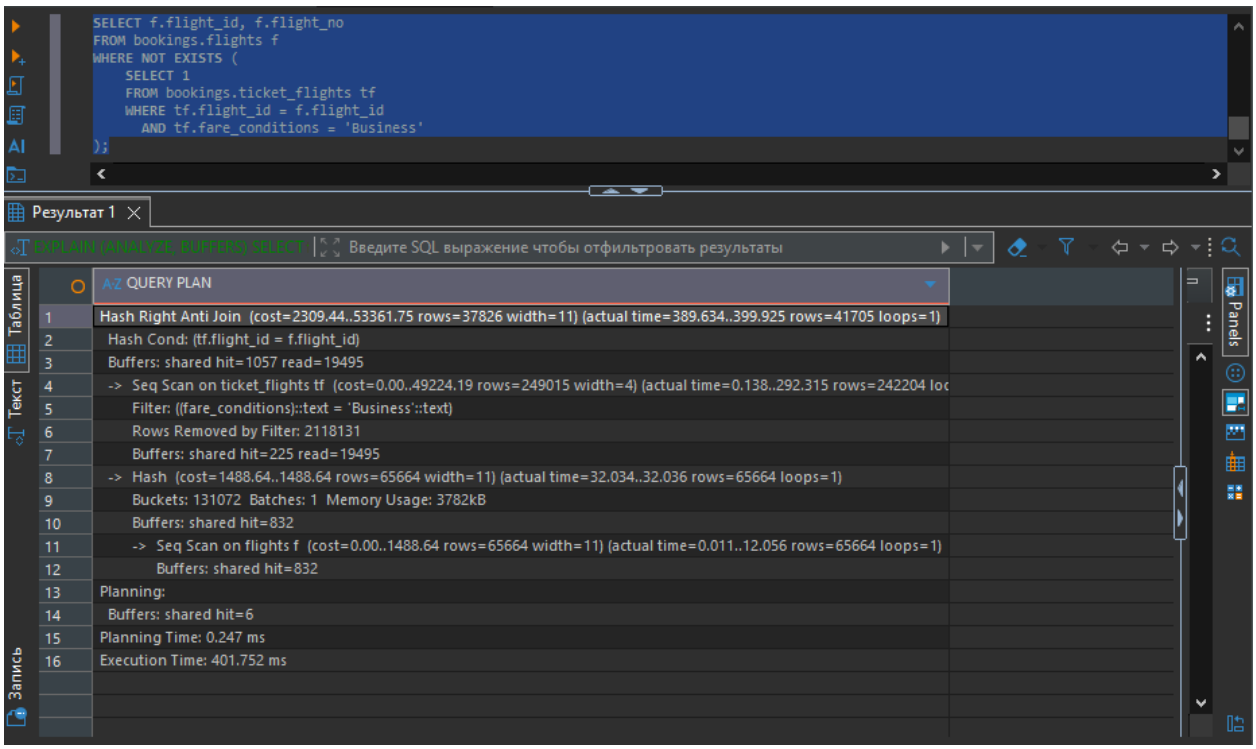
1 Вывод команды EXPLAIN (ANALYZE, BUFFERS) NOT IN**:**



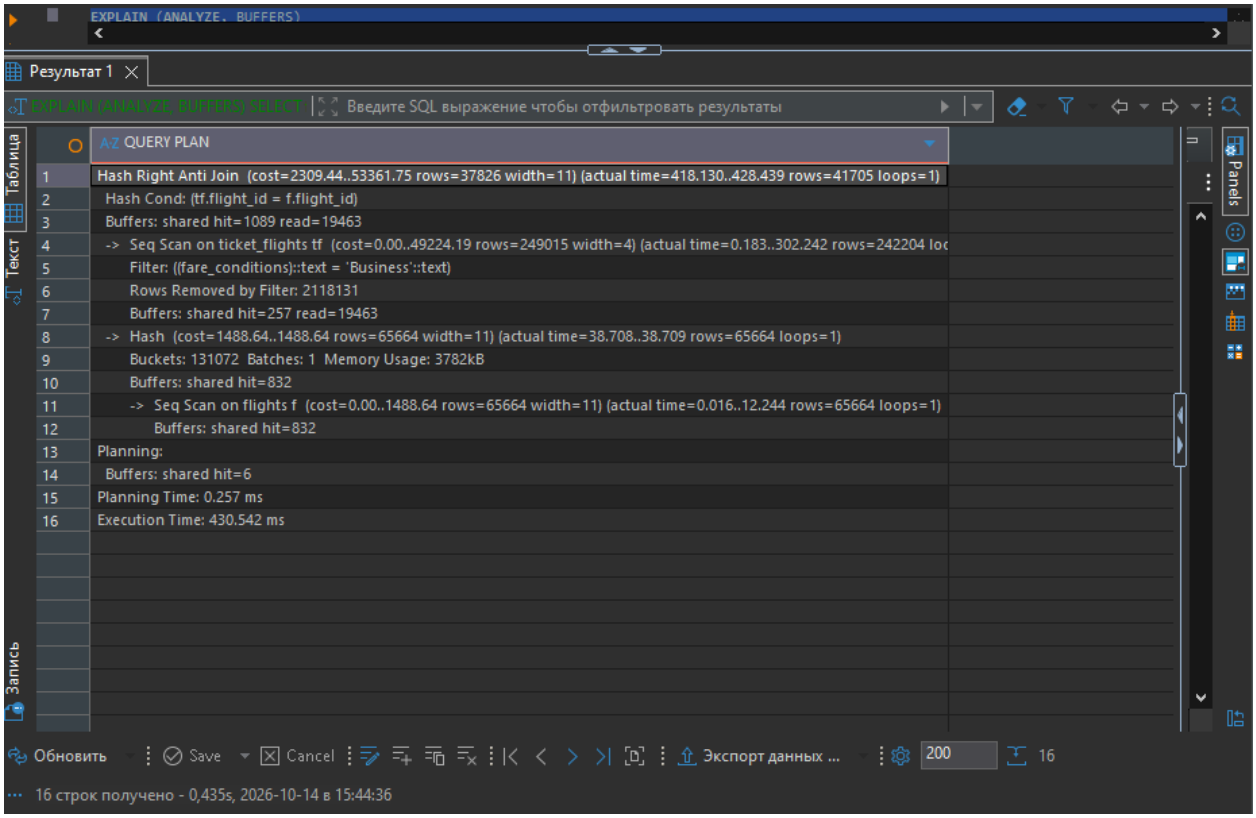
2 Вывод команды EXPLAIN (ANALYZE, BUFFERS) NOT IN:



1 Вывод команды EXPLAIN (ANALYZE, BUFFERS) NOT EXISTS:



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS) NOT EXISTS:



Аналитическое резюме:

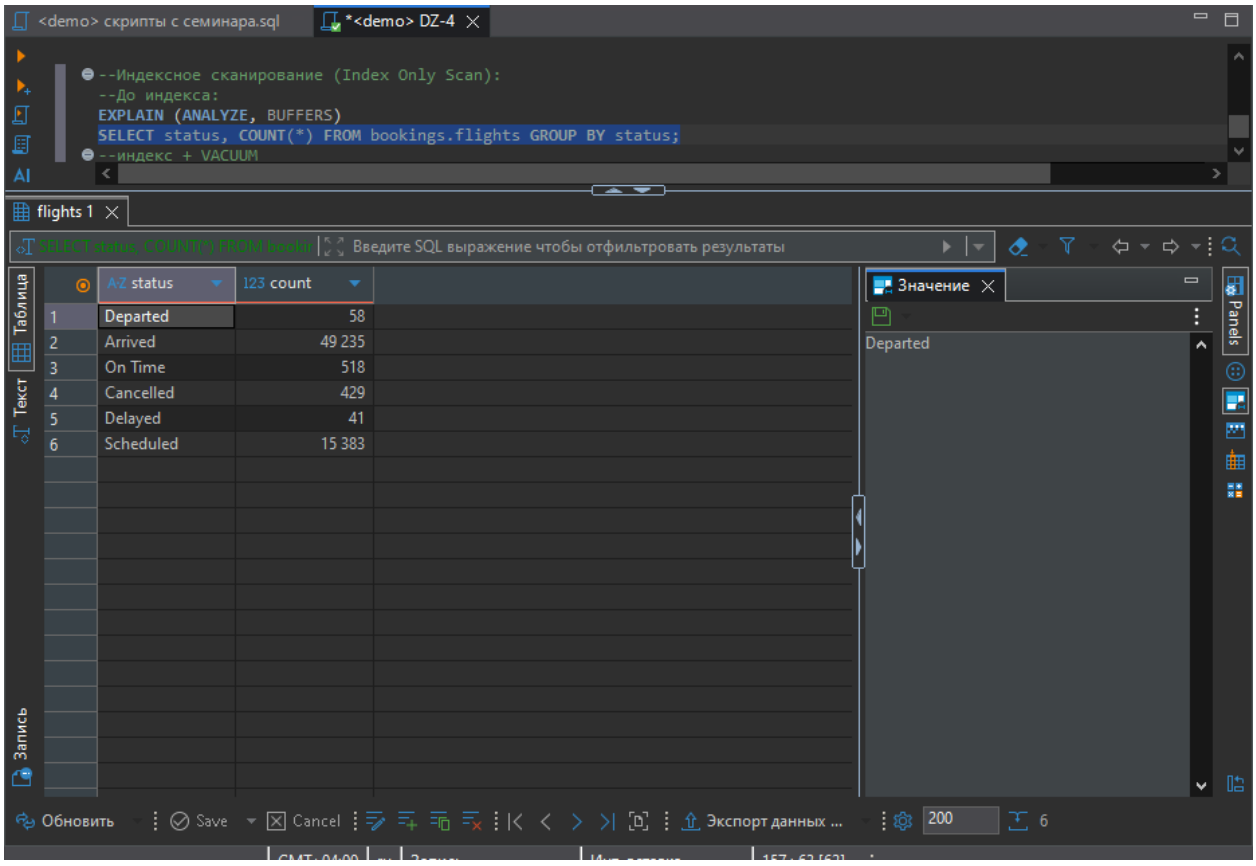
NOT IN: План - Seq Scan по flights с SubPlan 1 (хешированный подзапрос), внутри Seq Scan по ticket_flights с фильтром fare_conditions = 'Business'. Первый запуск hit=993, read=19559 (часть с диска, 463.9 мс), второй hit=1025, read=19527 (снова с диском, 391.2 мс). Индекс: по flight_id и fare_conditions в ticket_flights индекса нет, Seq Scan и SubPlan - единственный вариант. NOT IN потенциально опасен: если в подзапросе появится NULL, результат всего запроса станет пустым из-за трёхзначной логики.

NOT EXISTS: План - Hash Right Anti Join flights и ticket_flights по flight_id, внутри два Seq Scan (flights + ticket_flights с фильтром Business). Первый запуск hit=961, read=19591 (часть с диска, 719.8 мс), второй hit=1057, read=19495 (снова с диском, 430.5 мс). Индекс: по flight_id и fare_conditions индекса нет, Seq Scan - единственный вариант. Anti Join эффективнее NOT IN, потому что это одна операция соединения: строится хэш из подзапроса и одним проходом отбрасываются совпадения; NOT EXISTS не боится NULL, так как проверяет существование строки, а не сравнение значений.

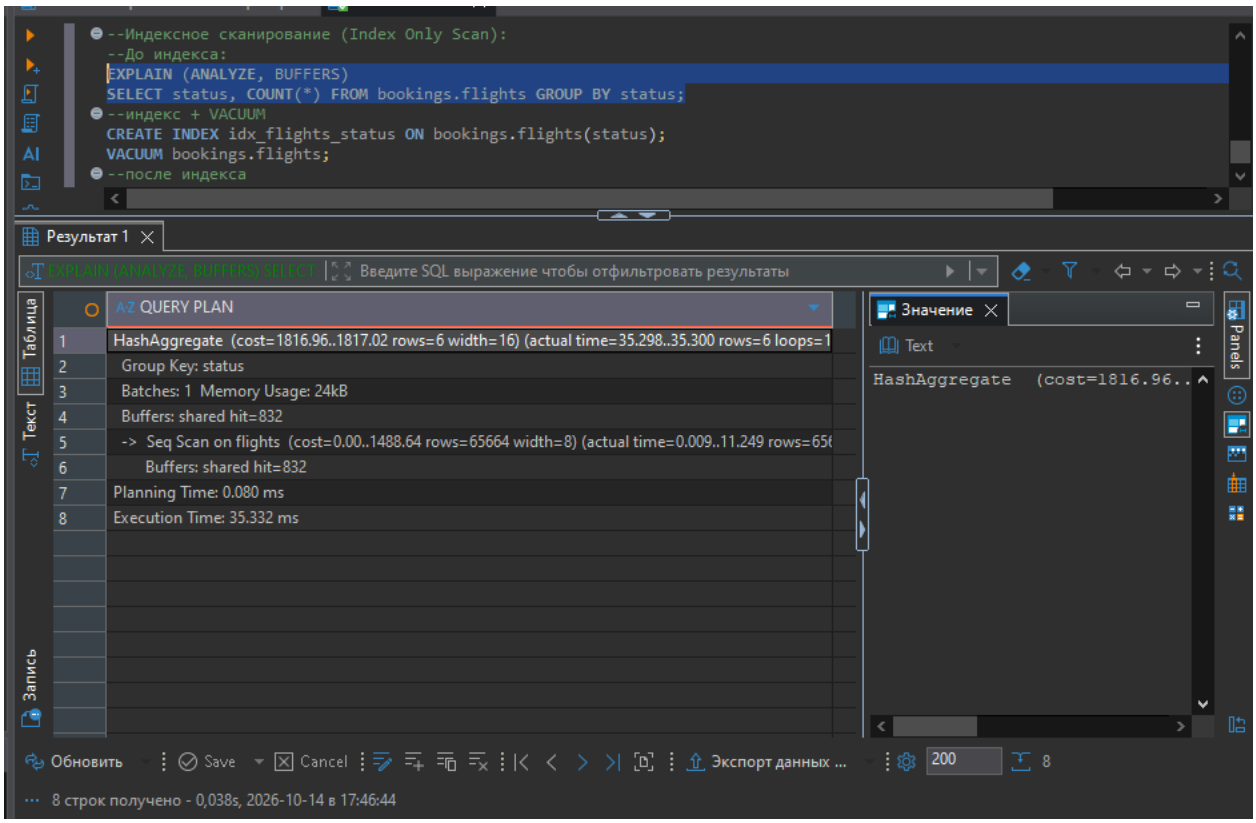
Индексное сканирование (Index Only Scan): Посчитайте количество рейсов для каждого статуса: SELECT status, COUNT(*) FROM flights GROUP BY status;

```
--До индекса:
EXPLAIN (ANALYZE, BUFFERS)
SELECT status, COUNT(*) FROM bookings.flights GROUP BY status;
--индекс + VACUUM
CREATE INDEX idx_flights_status ON bookings.flights(status);
VACUUM bookings.flights;
--после индекса
EXPLAIN (ANALYZE, BUFFERS)
SELECT status, COUNT(*) FROM bookings.flights GROUP BY status;
```

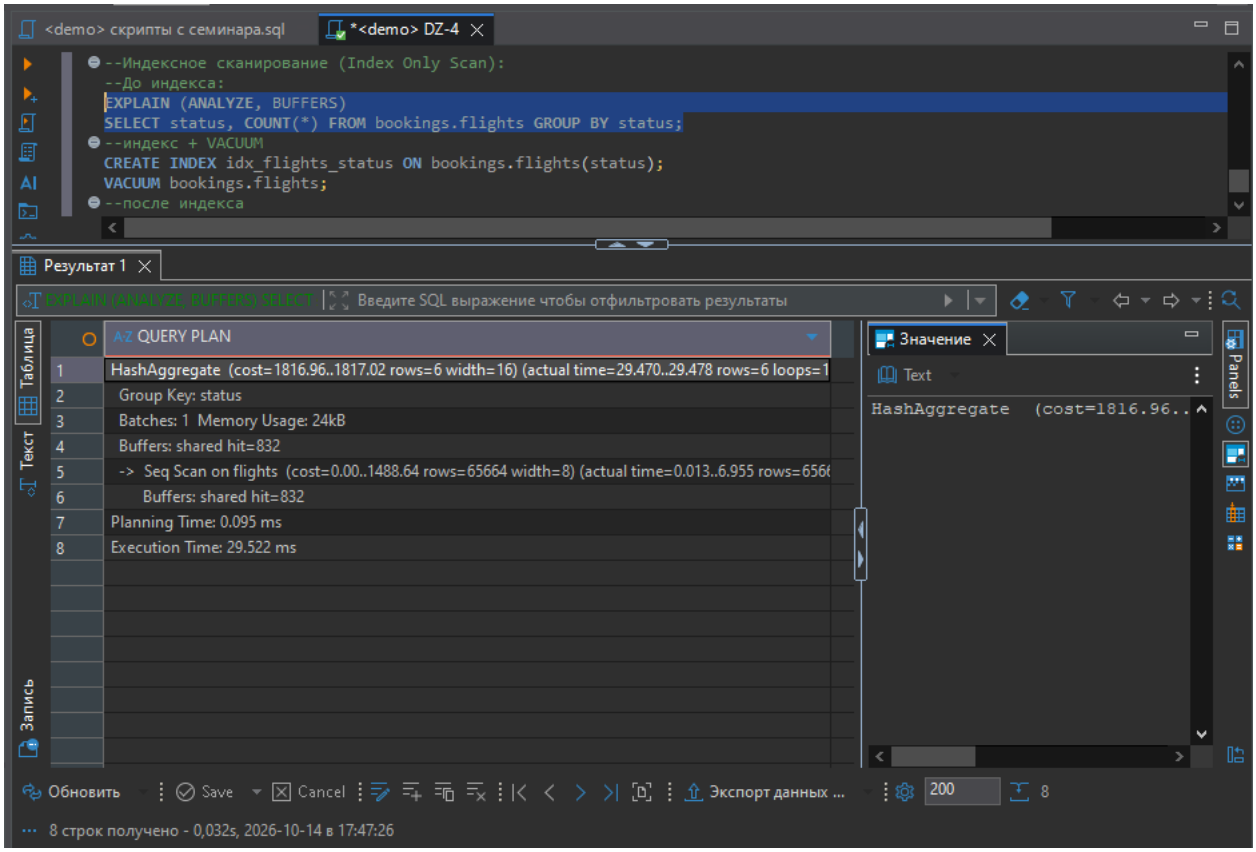
1 Вывод До индекса:



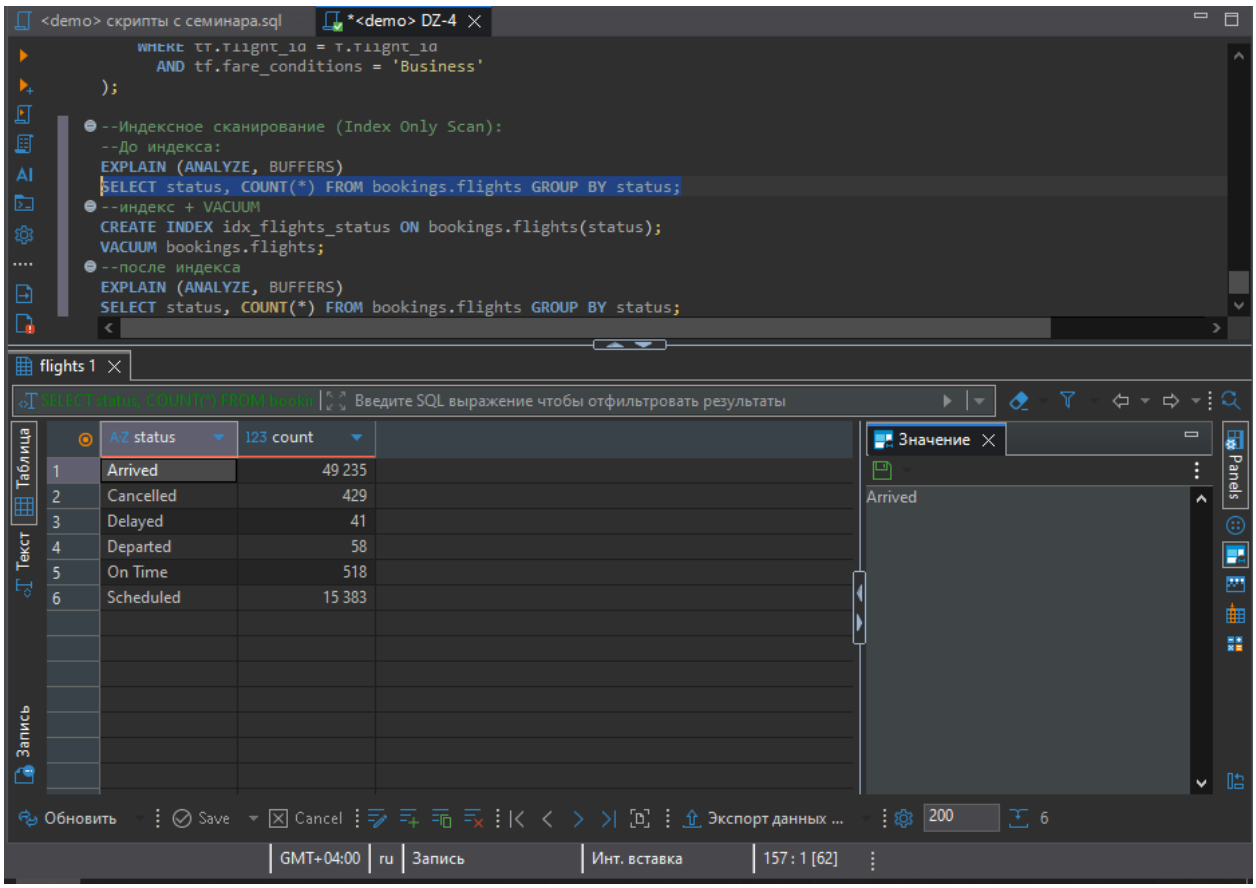
1 Вывод команды EXPLAIN (ANALYZE, BUFFERS) До индекса



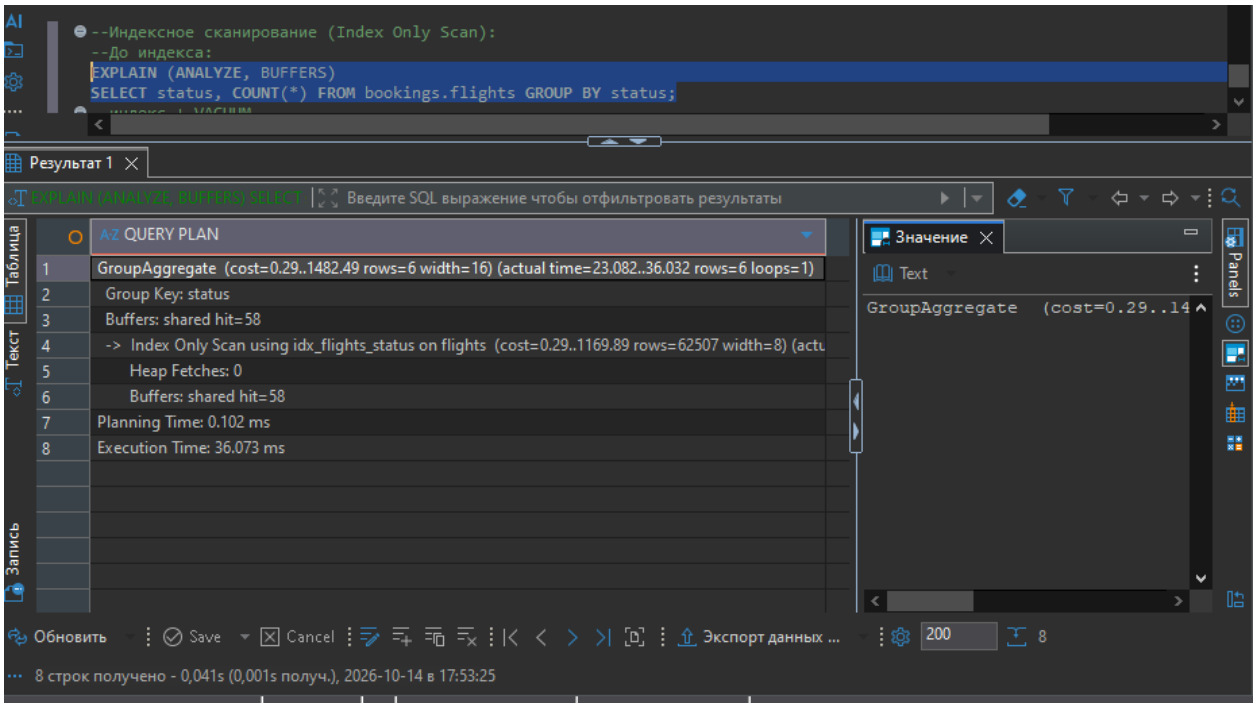
2 Вывод команды EXPLAIN (ANALYZE, BUFFERS) До индекса



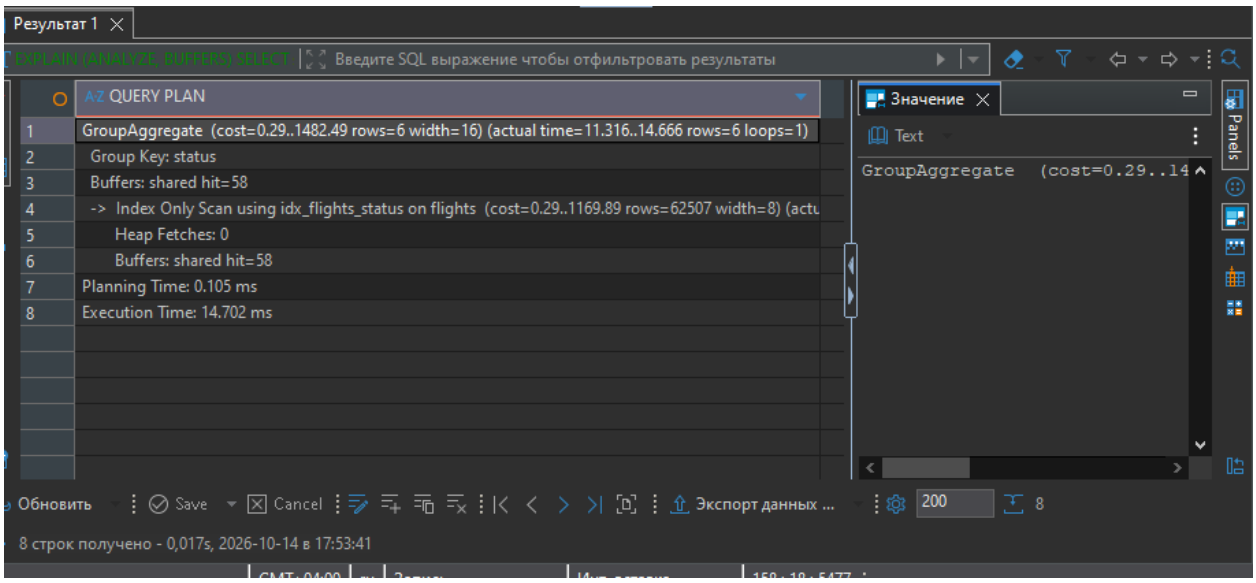
2 Вывод индекс + VACUUM



1 Вывод команды EXPLAIN (ANALYZE, BUFFERS) индекс + VACUUM



2 Вывод команды EXPLAIN (ANALYZE, BUFFERS) индекс + VACUUM



Аналитическое резюме:

до индекса: План - Seq Scan по flights, HashAggregate с группировкой по status, всё из кэша (hit=832, read=0), время 29.5–35.3 мс. Индекс: до создания idx_flights_status индекса по status не было, Seq Scan - единственный вариант.

после индекса + VACUUM: План Index Only Scan using idx_flights_status (Heap Fetches: 0), GroupAggregate, всё из кэша (hit=58, read=0), время 14.7–36.1 мс. Индекс: idx_flights_status использован, Index Only Scan работает без обращения к таблице. Heap Fetches: 0 означает, что VACUUM обновил Visibility Map и все страницы помечены как «all-visible» - СУБД берёт данные только из индекса. Буферов стало в 14 раз меньше (58 против 832), потому что индекс содержит только колонку status и меньше по размеру, чем таблица